First Lecture - Python Basics (assigning values to variables, different operator,New Identity and Membership operators)

```
#hello World
"""
multiline
commenting
"""
print("Hello World!")
```

```
Hello World!
```

```
#addition python code
a=10
b=10
c=a+b
print("Sum is ",c)
```

```
Sum is  20
```

```
#if else python code
if 5>2:
    print("5 > 2")
else :
    print("5 < 2")
```

```
5 > 2
```

```
#normal vraiable printing
n=4
print(n)
```

```
4
```

```
subject = "Klien Moretti"
classification ="Lord Of The Mysteries"
Danger ="Fighting Cosmic Corruption and Old Half Lord Of the
Mysteries"
Age = 24.25
Artifacts = 8
Sefirot ='Sefirah_Castle'
is_real=0+0j
print(f"Subject : {subject}")
print(f"classification : {classification}")
print(f"Danger : {Danger}")
print(f"Age : {Age}")
```

```
print(f"Artifacts : {Artifacts}")
print(f"is_real : {is_real}")
print(type(Age))
print(type(Artifacts))
print(type(Sefirot))
print(type(classification))
print(type(is_real))
```

```
Subject : Klien Moretti
classification : Lord Of The Mysteries
Danger : Fighting Cosmic Corruption and Old Half Lord Of the Mysteries
Age : 24.25
Artifacts : 8
is_real : 0j
<class 'float'>
<class 'int'>
<class 'str'>
<class 'str'>
<class 'complex'>
```

```
x=-36.978
y=12E78
print(type(x))
print(type(y))
```

```
<class 'float'>
<class 'float'>
```

```
x=True
print(x)
print(type(x))
```

```
True
<class 'bool'>
```

```
x=1
y=2.8
z=1j
a=float(x)
b=int(y)
c=complex(z)
print(a)
print(b)
print(c)
print(type(a))
print(type(b))
print(type(c))
print(f"a : {type(a)} : {a}")
print(f"b : {type(b)} : {b}")
print(f"c : {type(c)} : {c}")
```

```
1.0
2
1j
<class 'float'>
<class 'int'>
<class 'complex'>
a : <class 'float'> : 1.0
b : <class 'int'> : 2
c : <class 'complex'> : 1j
```

```
#arithmetic operators
a=5
b=3
print(a+b)#addition
print(a-b)#substraction
print(a*b)#multiplication
print(a/b)#division
print(a%b)#modulus
print(a//b)#floor division
print(a**b)#exponent
```

```
8
2
15
1.6666666666666667
2
1
125
```

```
#conditional operators
a=5
b=3
print(a>b)#greater then
print(a<b)#less then
print(a==b)#equal to
print(a!=b)#not equal to
print(a>=b)#greater then or equal to
print(a<=b)#less then or equal to
```

```
True
False
False
True
True
False
```

```
#bitwise operators
a=60
b=13
print(a & b)
```

```
print(a | b)
print(a ^ b)
print(~ a)
print(~ b)
print(bin(a))
print(bin(b))
```

```
12
61
49
-61
-14
0b111100
0b1101
```

```
a+=5
print(a)
```

```
65
```

```
#Membership operators
x="Hello World!"
print('H' in x)
print('Hello' not in x)
print('hello' in x)
```

```
True
False
False
```

```
#identity operator
x=["cat","mat"]
y=["cat","mat"]
z=x
print(x is z)
print(x is y)
print(x==y)
```

```
True
False
True
```

Input and Output in Python

```
#by default String
num = input("Enter a number : ")
print("You entered : ", num)
print("Data type of num : ", type(num))
```

```
Enter a number : 1
You entered :  1
Data type of num :  <class 'str'>
```

```
#typecasting input
a=int(input("Enter first no : "))
b=int(input("Enter second no : "))
sum = a+b
print("Sum is",sum)
```

```
Enter first no : 1
Enter second no : 2
Sum is 3
```

```
# print object,seperator and end
print("GFG")
print("G","F","G",sep="#",end="@!")
print("H&M")
print("H&MMM")
```

```
GFG
G#F#G@!H&M
H&MMM
```

```
print('New Year',2026,'Never see you again',sep='.')
```

```
New Year.2026.Never see you again
```

```
print('Science '+'is '+'Elegant')
```

```
Science is Elegant
```

```
print("Hello \t World")
print("Hello \\n World")
print(3*"Hello World")
```

```
Hello 	 World
Hello \n World
Hello WorldHello WorldHello World
```

```
a,b=1,2
print(a,b)
```

```
1 2
```

```
lst = [10,"A","hai"]
print(lst)
```

```
[10, 'A', 'hai']
```

```
d = {10:"Ram",20:"Amar"}
print(d)
```

```
{10: 'Ram', 20: 'Amar'}
```

```
print(f"The value of a is {a} and b is {b}")
print("The value of a is {} and b is {}".format(a,b))
```

```
The value of a is 1 and b is 2
The value of a is 1 and b is 2
```

ClassWork 1

```
# WAP to display your name and college name
name = "Aaryan"
c_name = "DJSCE"
print(name,c_name)
```

```
Aaryan DJSCE
```

```
# display designation
dsg = "Web Dev Intern"
name = "Aaryan"
c_name = "DJSCE"
print(dsg,name,"from",c_name)
```

```
Web Dev Intern Aaryan from DJSCE
```

```
pi = float(3.14)
r = int(2)
area = pi * (r ** 2)
print(area)
```

```
12.56
```

```
pi = float(3.14)
r = int(2)
area = pi * (r ** 2)
print(int(area))
```

```
12
```

```
pi = float(3.14)
r = int(input("Enter Radius : "))
```

```
area = pi * (r ** 2)
print(int(area))
```

```
Enter Radius : 3
28
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab (DJS23SLPC303)
Second Year B. Tech Semester: III

PRACTICAL 1

Part A (To be referred by students)

Data Types, Input / Output Statements, Operators (Arithmetic, Relational, Logical, Bitwise) and Expressions

Aim: To implement program to demonstrate data types operator and input output in python.

Problem Statements:

1. Create a program that takes user input and determines its data type.
2. Write a program to enter a number and print its square, cube and square root.
3. Write a program to perform arithmetic operations on two numbers.
4. Create a program that uses all comparison operators and prints the results.
5. Write a program to calculate the area of a right-angle triangle.
6. Calculate average of 3 numbers.
7. Calculate simple interest.
8. Convert Celsius to Fahrenheit.
9. Find Euclidean distance between two points.
10. Swap two variables using: Temp variable, Comma operator, Bitwise XOR
11. Find roots of a quadratic equation $ax^2 + bx + c = 0$.

Learning Objective: The Learner will be able to

1. Analyze the scenario to write a script declaring variables, printing output and using expressions.
2. Infer the features of various operators for different operations as per variable data type.
3. Evaluate the working of the algorithm using variables, printing and inputting user values and expressions.

Theory:

- **Data types:**
  Number: int, float, complex
  String

```
a = 11.80
x  = 100
y = "Hello"
z = 3+5.j

print(a)
print(x)
print(y)
print(z)
```

Output:

```
11.8
100
Hello
(3+5j)
```

- **Accepting Input from the Console:**

```
# if input() used it accepts input as string
num=input('Enter a number: ')
print(num)

# to accept numbers
num=int(input('Enter a number: '))
print(num)

num=float(input('Enter a number: '))
print(num)
```

Output:
Enter a number: 10
'10'
Enter a number: 45
45
Enter a number: 78
78.0

- **Output using print() with formatting:**

```
x=6
y=89
print('The value of x is {1} and y is {0}'.format(x,y))
x = 12.3456789
print('The value of x is %3.2f' %x)

print('The value of x is %3.4f' %x)

print ('The %.2f and %.1f ' %(x, y))
```

Output:
The value of x is 89 and y is 6
The value of x is 12.35
The value of x is 12.3457
The 12.35 and 456.8

```
name = "Alice"
age = 25
print(f"{name} is {age} years old.")
```

Output:
Alice is 25 years old.

- **Operators:**

Operators are used to perform operations on variables and values.
Python divides the operators in the following groups:

- **Arithmetic operators**
- Assignment operators
- **Relational/Comparison operators**
- **Logical operators**
- Identity operators
- Membership operators
- **Bitwise operators**

- **Python Arithmetic Operators**

Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Example
+	**x + y**
-	**x - y**
*	**x * y**
/	**x / y**
%	**x % y**
**	**x ** y**
//	x // y

- **Python Assignment Operators**

Assignment operators are used to assign values to variables:

Operator	Example

=	x = 5
+=	x += 3
-=	x -= 3
*=	x *= 3
/=	x /= 3
%=	x %= 3
//=	x //= 3
**=	x **= 3
&=	x &= 3
|=	x |= 3
^=	x ^= 3
>>=	x >>= 3
<<=	x <<= 3

- **Python Relational/Comparison Operators**

Comparison operators are used to compare two values:

Operator	Example
==	x == y
!=	x != y
>	x > y
<	x < y
>=	x >= y
<=	x <= y

- **Python Logical Operators**

Logical operators are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

- **Python Bitwise Operators**

Bitwise operators are used to compare (binary) numbers:

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

PRACTICAL 1

Part B (to be completed by students)

Data Types, Input / Output Statements, Operators (Arithmetic, Relational, Logical, Bitwise) and Expressions

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp1)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments

4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp1.pdf
6. Upload the pdf on the web portal (Final Submission)
7. Submit the handwritten experiment in the next lab session

```python
#Determine user input
something = input("Enter something : ")
print("You entered : ", something)
print("Data type of num : ", type(something))
```

```
Enter something : fhsjigvhuhb
You entered :  fhsjigvhuhb
Data type of num :  <class 'str'>
```

```python
num = int(input("Enter a number : "))
print("Square : ",(num ** 2))
print("Cube : ",(num ** 3))
print("Square root : ",(num ** 0.5))
```

```
Enter a number : 4
Square :  16
Cube :  64
Square root :  2.0
```

```python
#arithmentic operation
a=int(input("Enter first no : "))
b=int(input("Enter second no : "))
print("addition is : ",(a+b))
print("substraction is : ",(a-b))
print("multiplication is : ",(a*b))
print("division is :",(a/b))
print("modulus is : ",(a%b))
print("floor division is : ",(a//b))
print("exponent is : ",(a**b))
```

```
Enter first no : 1
Enter second no : 2
addition is :  3
substraction is :  -1
multiplication is :  2
division is : 0.5
modulus is :  1
floor division is :  0
exponent is :  1
```

```python
#comparison operator
a=int(input("Enter first no : "))
b=int(input("Enter second no : "))
print((a>b))#greater then
print((a<b))#less then
print((a==b))#equal to
print((a!=b))#not equal to
print((a>=b))#greater then or equal to
print((a<=b))#less then or equal to
```

```
Enter first no : 2
Enter second no : 3
False
True
False
True
False
True
```

```
#area of right angled triangle
l=float(input("Enter length : "))
h=float(input("Enter breadth : "))
print("Area of the right angled triange : ",(0.5*l*b))
```

```
Enter length : 2
Enter breadth : 3
Area of the right angled triange :  3.0
```

```
# average
a=float(input("Enter first no : "))
b=float(input("Enter second no : "))
c=float(input("Enter third no : "))
avg = (a+b+c)/3
print("Average : ",avg)
```

```
Enter first no : 1
Enter second no : 2
Enter third no : 3
Average :  2.0
```

```
#simple interest
p=float(input("Enter principal value : "))
r=float(input("Enter interest rate : "))
n=float(input("Enter no of year: "))
si = (p*r*n)/100
print("Simple Interest : ",si)
```

```
Enter principal value : 1000
Enter interest rate : 5
Enter no of year: 3
Simple Interest :  150.0
```

```
# celsius to fahrenheit
c = float(input("Enter the temperature in celsius : "))
f = ((9/5)*c)+32
print("Temperature in Fahrenheit : ",f)
```

```
Enter the temperature in celsius : 29
Temperature in Fahrenheit :  84.2
```

```
# euclidean distance
x1 = float(input("Enter first x co-ordinate : "))
y1 = float(input("Enter first y co-ordinate : "))
x2 = float(input("Enter second x co-ordinate : "))
y2 = float(input("Enter second y co-ordinate : "))
dist = ((((x2-x1)**2)+((y2-y1)**2)) ** 0.5)
print("Euclidean distance : ",dist)
```

```
Enter first x co-ordinate : 1
Enter first y co-ordinate : 2
Enter second x co-ordinate : 3
Enter second y co-ordinate : 4
Euclidean distance :  2.8284271247461903
```

```
# swap 2 variables using comma,temp,bitwiseXOR
a=3
b=4
a,b=b,a
print(a,b)
```

```
4 3
```

```
a=3
b=4
t=a
a=b
b=t
print(a,b)
```

```
4 3
```

```
a=9
b=4
a= a^b
b = a^b
a = a^b
print(a,b)
```

```
4 9
```

```
# solution to quadratic equation
a=float(input("Enter first co-effiecient : "))
b=float(input("Enter second co-effiecient : "))
c=float(input("Enter third co-effiecient : "))
D = ((b**2)-(4*a*c))
if D > 0:
    x1=(((b*(-1))+(D ** 0.5))/(2*a))
    x2=(((b*(-1))-(D ** 0.5))/(2*a))
    print("Roots of the equation are : ",x1,"and",x2)
elif D==0:
    x=((b*(-1))/(2*a))
```

```
    print("Root of the equation is : ",x)
else:
    print("Real solution doesn't exist")
    x1=(((b*(-1))+(D ** 0.5))/(2*a))
    x2=(((b*(-1))-(D ** 0.5))/(2*a))
    print("Complex Roots of the equation are : ",x1,"and",x2)
```

```
Enter first co-effiecient : 1
Enter second co-effiecient : 1
Enter third co-effiecient : 1
Real solution doesn't exist
Complex Roots of the equation are :  (-
0.49999999999999994+0.8660254037844386j) and (-0.5-
0.8660254037844386j)
```

Name :- Aaryan N. Rodrane Roll no :- 5004
branch :- AI-DS batch : A1 SAPID :- 60018250070

Experiment No 1

Title :- Data-Types, operators and input-output in Python

Aim :- To implement a program to demonstrate data types operator and input output in Python

Theory :-

Basic Data types in Python with Examples

Python is dynamically typed, interpreted language. Unlike statically typed languages such as C or Java, Python determines the data type of a variable at runtime based on the value assigned to it. Every value in Python is an object, and each object belongs to a specific class.

Python provides the following built-in data types:

1. **Int (Integer) :**

The int type stores whole numbers without a decimal component. Python integers have arbitary precision, meaning they are not constrained by a fixed bit width like 32-bit or 64-bit integers in other languages.

FOR EDUCATIONAL USE

code:
```
x = 10
y = -25
z = 0
print (type(x)) # <class 'int'>
print (x + y) # [output: -15]
```

2. float (floating-point Number)

The float type represents real numbers with a decimal point. Internally, Python floats follow the IEEE 754 double-precision standard, which provides approximately 15 to 17 significant decimal digits of precision.

code:
```
pi = 3.14159
temperature = -7.5
print (type(pi)) # output: <class 'float'>
print (pi * 2) # output: 6.28318
```

3. complex

The complex type stores numbers of the form $a+bj$, where a is the real part and b is the imaginary part. Python uses j to denote the imaginary unit.

code:
```
c = 3 + 4j
print (c.real) # output: 3.0
print (c.imag) # output: 4.0
print (type(c)) # output: <class 'complex'>
```

Sundaram
FOR EDUCATIONAL USE

4. str (string)

A string is an ordered, immutable sequence of Unicode characters. Strings can be enclosed in single quotes, double quotes or triple quotes (for multiline strings).

code :
```
name = "Python"
message = 'Hello, World!'
multiline = """This is a
multiline string."""
print (type(name)) # output : <class 'str'>
print (name[0]) # output : P
print (len(name)) # output : 6
```

5. Bool (Boolean)

The Bool type holds one of two values : True or False. Booleans are technically a subclass of int in Python, where True evaluates to 1 and False evaluates to 0

code :
```
is_active = True
is_empty = False
print (type(is_active)) # output : <class 'bool'>
print (is_active + 6) # output : 7
print (int(is_empty)) # output : 0
```

Sundaram
FOR EDUCATIONAL USE

*) Basic Operators in Python with Examples

Operators are tokens that instruct the interpreter to perform a specific computation on one or more operands. Python supports several categories of operators.

1. Arithmetic Operator

These operators perform standard mathematical computations. The floor division operator (//) returns the quotient rounded down to the nearest integer, and the modulus operator (%) returns the remainder.

code:-

```
a = 10
b = 3

print(a+b) # Addition output: 13
print(a-b) # Substraction output: 7
print(a*b) # Multiplication output: 30
print(a/b) # Division output: 3.3333333....
print(a//b) # floor Division output: 3
print(a%b) # Modulus Output: 1
print(a**b) # Exponent output: 1000
```

Sundaram
FOR EDUCATIONAL USE

2. Comparison (Relational) operators

Comparison operators evaluate the relationship between two operands and return a Boolean value (True or False). These are widely used in conditional statements and loop control.

Code:-

```
x = 5
y = 10

print(x == y) # equal to output : False
print(x != y) # Not equal to output : True
print(x > y) # greater than output : False
print(x < y) # Less than output : True
print(x >= y) # greater or equal to output : False
print(x <= y) # Less or equal to output : True
```

3. Assignment operators

Assignment operators binds value to a variable. Compound assignment operators (such as += and *=) combine an arithmetic operation with assignment into a single statement.

Code:-

```
a = 10
a += 5 # Equivalent to a = a+5 → a = 15
a -= 3 # Equivalent to a = a-3 → a = 12
a *= 2 # Equivalent to a = a*2 → a = 24
a //= 4 # Equivalent to a = a//4 → a = 6
print(a) # output : 6
```

Sundaram
FOR EDUCATIONAL USE

4. Logical operators

Logical operators combine or negate Boolean expressions. Python evaluates and and or using short-circuit evaluation: it stops evaluating as soon as the result is determined.

code:
```
p = True
q = False

print(p and q) # both must be True output: False
print(p or q) # at least one true output: True
print(not p) # Negation output: False
print(not q) # Negation output: True
```

5. Bitwise operators

Bitwise operators operate directly on the Binary representation of integer operands. They are used in low-level programming, cryptography and performance-sensitive applications.

code:
```
a = 5    # binary : 0101
b = 3    # binary : 0011

print(a & b) # AND output: 1 (0001)
print(a | b) # OR output: 7 (0111)
print(a ^ b) # XOR output: 6 (0110)
print(~a) # NOT output: -6 (1010)
print(a << 1) # left shift output: 10 (1010)
print(a >> 1) # Right shift output: 2 (0010)
```

Sundaram
FOR EDUCATIONAL USE

5. Membership Operators

Membership operators test whether a value is present in a sequence such as a list, tuple, string or set. They return a Boolean result.

Code:
```
fruits=["apple","Banana","mango"]

print("apple" in fruits) # output: True
print("grape" in fruits) # output: False
print("grape" not in fruits) # output: True

text = "Python Programming"
print("Python" in text) # output: True
```

6. Identity Operators

Identity operators check whether two variables refer to the same object in memory. This is different from equality: two objects can hold the same value but occupy different memory locations.

Code:
```
a=[1,2,3]
b=a # b refers to the same object as a
c=[1,2,3] # c is a new object with the same value
print(a is b) # output: True (same object in memory)
print(a is c) # output: False (different objects)
print(a==c) # output: True (same values)
print(a is not c) # output: True
```

Sundaram
FOR EDUCATIONAL USE

3. Input and Output in Python

Python provides built in functions for reading data from the user and displaying results to the console.

* Output using print()

The print() function writes data to the standard output stream. It accepts multiple positional arguments and two keywords: sep (separator between arguments, default is a space) and end (character printed after the last argument, default is a newline)

Syntax: print (object(s), sep=' ', end='\n')

code:
```
print ("Hello, World!") # output: Hello, World!
print (10, 20, 30)  # output: 10 20 30
print (10, 20, 30, sep=":", ) #output: 10, 20, 30
print ("First", end = " ")
print ("second") # output: First second
print ("Name:", "Carlson", "Age:", 20)
    # output: Name: Carlson Age: 20
```

* formatted output

Python supports three methods for formatted string output:

Sundaram
FOR EDUCATIONAL USE

Method 1 :- % formatting (old style)

code :-
```
name = "Aaryan"
marks = 95.5
print ("student : %s, Marks : %.2f" %
       (name, marks))
#output : student : Aaryan , Marks : 95.50
```

Method 2 :- str.format() method

code :-
```
print ("Student : {} , Marks : {:.2f}".format(name,
       marks))
#output : student : Aaryan, Marks : 95.50
```

Method 3 :- f-strings (Python 3.6 and above)

code :-
```
print (f "Student : {name}, Marks : {marks:.2f}")
#output : Student : Aaryan , Marks : 95.50
```

f-strings are the most readable and efficient of the three methods. they allow expressions to be embedded directly inside the string literal

* Input using input()

The input() function pauses program execution and reads a line of text from the standard input stream. It always return the value as a string (str), regardless of what the user types.

Sundaram
FOR EDUCATIONAL USE

Syntax: input ([prompt])

code:
```
name = input("Enter your name: ")
print("Hello,", name)
# input : Karl
# output : Hello, Aaryan.
```

* Type Conversion (casting)

Since input() returns a string, numeric input must be explicitly converted using type-casting functions.

code:
```
age = int(input("Enter your age: "))
cgpa = float(input("Enter your CGPA: "))
print(f"Age: {age}, CGPA: {cgpa:.2f}")
```

Python's built-in type conversion functions:

1. int(x) : converts x to an integer
2. float(x): converts x to a floating-point number
3. str(x) : converts x to a string.
4. bool(x): converts x to a Boolean
5. complex(x): converts x to a complex number.

code:
```
x = "42"
print(int(x))  # output: 42
print(float(x)) # output: 42.0
print(bool(0)) # output: False
print(bool(99)) # output: True
print(str(3.14)) # output: '3.14'
```

Sundaram
FOR EDUCATIONAL USE

* Python input - output code :-

```
length = float(input("Enter length : "))
breadth = float(input("Enter breadth : "))
area = length * breadth
perimeter = 2 * (length + breadth)

print(f"\n Rectangle Details")
print(f"length : {length}")
print(f" breadth : {breadth}")
print(f"Area : {area :.2f} sq. units")
print(f"Perimeter : {perimeter :.2f} units")
```

output: Enter length : 5.0
Enter breadth : 3.0

Rectangle Details
length : 5.0
breadth : 3.0
Area : 15.00 sq. units
Perimeter : 16.00 units

* Conclusion :-

Through this experiment, the Basic data types in Python such as int, float, complex, str and bool were studied along with their usage in programs. The various operators like arithmetic, logical, relational and bitwise were understood and implemented with examples. The input() and print() functions were used to handle user input and display formatted output on the console.

Sundaram
FOR EDUCATIONAL USE

Control Statement (If else , switch ,for , while, break,etc.)

```
# if else with indentation error
a=23
b=302
if a>b:
    print("b is greater then a")
else:
print("a is greater then b")
```

```
  File "<ipython-input-1-990ba1a9f7f2>", line 7
    print("a is greater then b")
        ^
IndentationError: expected an indented block
```

```
# if else
a=23
b=302
if a>b:
    print("b is greater then a")
else:
    print("a is greater then b")
```

```
a is greater then b
```

```
msg = input("Are willing to learn Python 'yes' or 'no' : \n")
if msg=="yes":
    print("Welcome to Python Programming")
```

```
Are willing to learn Python 'yes' or 'no' :
yes
Welcome to Python Programming
```

```
#using only if
a=int(input("Enter no. 1 : "))
b=int(input("Enter no. 2 : "))
if a>b:
    print("no. 1 is greater then no. 2")
if a==b:
    print("no. 1 is equal to no. 2")
if b>a:
    print("no. 2 is greater then no. 1")
```

```
Enter no. 1 : 10
Enter no. 2 : 56
no. 2 is greater then no. 1
```

```
#using only if and else
a=int(input("Enter no. 1 : "))
b=int(input("Enter no. 2 : "))
if a>b:
    print("no. 1 is greater then no. 2")
if a==b:
    print("no. 1 is equal to no. 2")
else:
    print("no. 2 is greater then no. 1")
```

```
Enter no. 1 : 10
Enter no. 2 : 10
no. 1 is equal to no. 2
```

```
#using only if and else
a=int(input("Enter no : "))
if a>0:
    print("Positive")
if a==0:
    print("Zero")
else:
    print("Negative")
```

```
Enter no : -88
Negative
```

```
#using only if and else
a=int(input("Enter no : "))
if a%2==0 :
    print("Even")
else:
    print("Odd")
```

```
Enter no : 45646478313
Odd
```

```
#using only if , elif and else
a=int(input("Enter age : "))
if a>=18:
    print("Eligible to Vote")
elif a>0:
    print("Not eligible to Vote")
else:
    print("U are not a human !! Go away Demon !! (Splashing Holy water
on you)")
```

```
Enter age : -9999
U are not a human !! Go away Demon !! (Splashing Holy water on you)
```

```
a=int(input("Enter no. 1 : "))
b=int(input("Enter no. 2 : "))
```

```
c=int(input("Enter no. 3 : "))
if a>b and a>c:
    print(f"{a} is greatest")
elif a>c and b>a:
    print(f"{b} is greatest")
else:
    print(f"{c} is greatest")
```

```
Enter no. 1 : 10
Enter no. 2 : 50
Enter no. 3 : 13
13 is greatest
```

```
a=int(input("Enter no : "))
if a>0:
    print("Positive")
elif a==0:
    print("Zero")
else:
    print("Negative")
```

```
Enter no : -9
Negative
```

```
# nested if-else
a = int(input("Enter no. : "))
if a> 10:
    print("Above 10")
    if a>20:
        print("And also above 20")
    else :
        print("but not above 20")
```

```
Enter no. : 41
Above 10
And also above 20
```

```
# one line if-else
a=23
b=24
print("A") if a>b else print("=") if a==b else print("B")
```

```
B
```

Strings and string operations

```
a = "This is name1"
b = 'This is name2'
```

```
print(a)
print(b)
```

```
This is name1
This is name2
```

```
a = "This is 'name1'"
print(a)
b = 'This is "name2"'
print(b)
```

```
This is 'name1'
This is "name2"
```

```
# backslash character
a = "\"hello\" This isn\'t name1"
print(a)
```

```
"hello" This isn't name1
```

```
# accessing string character
print(a[0])
print(a[5])
print(a[-3])
```

```
"
o
m
```

```
# slicing
print(a[4:-11])
print(a[4:])
print(a[:-11])
print(a[-11:-4])
```

```
lo" This
lo" This isn't name1
"hello" This
isn't n
```

```
#concatenation
x = "Helllo"
y = "World"
z = "I"
a = "am"
b = "???"
print(a + b + y + z + x)
# string repition
print(3*(a + b + y + z + x))
```

```
am???WorldIHelllo
am???WorldIHellloam???WorldIHellloam???WorldIHelllo
```

```
# == operator
print("Hello" == "Hello")
print("Hello" == "hello")
```

```
True
False
```

```
# != operator
print("Hello" != "Hello")
print("Hello" != "hello")
```

```
False
True
```

```
# > operator
print("Hello" > "Hello")
print("Hello" > "hello")
```

```
False
False
```

```
# < operator
print("Hello" < "Hello")
print("Hello" < "hello")
```

```
False
True
```

```
print('jas' in 'jaspreet')
```

```
True
```

```
print('jas' not in 'jaspreet')
```

```
False
```

```
age = int(31)
print("age is %d"%(age))
```

```
age is 31
```

```
s='I am Jaspreet'
print(s[0:8])
```

```
I am Jas
```

```
s='abc'*3
print(s)
print(s[0:6:2])
print(s[0:6:2])
print(s[2:10:2])
print(s[::3])
print(s[1::3])
```

```
print(s[:8:3])

#3rd parameter as negative index(1st parameter then must be greater
then 2nd parameter)

print(s[8:1:-2])
print(s[8:1:-1])
print(s[8:-1:-1])
print(s[::-1])
```

```
abcabcabc
acb
acb
cbac
aaa
bbb
aaa
cabc
cbacbac

cbacbacba
abcabcabc
```

Strip Method

```
s="          abcabcabc      "
print(s.strip())
s="aaaaaaaabcabcabcaaaa"
print(s.strip('a'))
s="Hello world"
print(s.strip('ldoH'))
print(s.strip('ldoh'))
```

```
abcabcabc
bcabcabc
ello wor
Hello wor
```

Lstrip and Rstrip Method

Removes only leading and trailing whitespaces by default respectively

```
s="         Hello world      "
print(s.lstrip())
print(s.rstrip())
```

```
Hello world
         Hello world
```

```
"  solving any problem is an art.  ".strip('. ')
```

```
'solving any problem is an art'
```

```
"  solving any problem is an art.  ".strip(' ')
```

```
'solving any problem is an art.'
```

```
str="I am Jaspreet      ."
str = str.rstrip('I am.')
str
```

```
'I am Jaspreet'
```

```
str="I am Jaspreet      ."
str = str.lstrip('I am.')
str
```

```
'Jaspreet      .'
```

```
sentence ="My name is Jaspreet  ".rstrip("my ")
sentence
```

```
'My name is Jaspreet'
```

Split method

syntax : string.split(separator,maxsplit)

```
"Hell$I$am$Jaspreet".split('$',maxsplit=2)
```

```
['Hell', 'I', 'am$Jaspreet']
```

```
"Hell$I$am$Jaspreet".rsplit('$',maxsplit=2)
```

```
['Hell$I', 'am', 'Jaspreet']
```

```
"Hello world! I am tired saying this sentence again and again".split()
```

```
['Hello',
 'world!',
 'I',
 'am',
 'tired',
 'saying',
 'this',
 'sentence',
 'again',
 'and',
 'again']
```

Join method

Syntax : separator.join(iterator)

```
s = "Hello world! I am tired saying this sentence again and
again".rsplit()
print(s)
s='    '.join(s)
print(s)
```

```
['Hello', 'world!', 'I', 'am', 'tired', 'saying', 'this', 'sentence',
'again', 'and', 'again']
Hello    world!    I    am    tired    saying    this    sentence
again    and    again
```

```
s=s.split()
print(s)
```

```
s='_@_'.join(s)
print(s)
```

```
['Hello', 'world!', 'I', 'am', 'tired', 'saying', 'this', 'sentence',
'again', 'and', 'again']
Hello_@_world!
_@_I_@_am_@_tired_@_saying_@_this_@_sentence_@_again_@_and_@_again
```

```
s='aBc'
print(s.upper())
print(s.lower())
print(s.capitalize())
```

```
ABC
abc
Abc
```

```
s='Python is Fun'
print(s.replace('Py','Kr'))
print(s.count('n'))
print(len(s))
print(s.find('is'))
```

```
Krthon is Fun
2
13
7
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 2

Part A (To be referred by students)

Conditional Statements & Strings

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp2)

Aim: To Implement python programs to demonstrate different decision-making and Control Statements and string handling.

Problem Statement: Write Python program to

1. Accept three sides of a triangle and print if the triangle is equilateral, isosceles or scalene.
2. A function f is defined as follows :

   $f(x) = ax^3 - bx^2 + cx - d$, if $x > k$
   $= 0$, if $x = k$
   $= -ax^3 + bx^2 - cx + d$, if $x < k$

   Write a program that reads a, b, c, d, k and x and prints the value of f(x).
3. Implement Body Mass Index calculator. Define BMI function to accept weight and height as parameter and displays appropriate BMI status.
   BMI = weight(kg)/Height^2

BM	Status
<= 18.4	Underweight
18.5 – 24.9	Normal
25.0 – 39.9	Overweight
>=40	Obese

4. Implement distance convertor to convert distances by reading choice from user
   1. inches to feet
   2. cm to meter
5. Write a program to read and display a string
6. Create a string variable s with the value "Hello World" and print its length.
7. Convert the string "Python Programming" to lowercase and print it.
8. Concatenate the strings "Data" and "Science" with a space in between and print the result.
9. Remove all leading and trailing whitespace from the string " Data Analysis " and print it.
10. Define one string and display all elements at odd indexes using slicing operator
11. Write a program to check entered string is palindrome or not.
12. Reverse the string 'Hello' only using string slicing
13. WAP to split the string into a list of words and then join them back into a single string with a user-defined delimiter.
14. Write a python program to take a list of words, reverse the order of the words, and then join them into a single string with a hyphen - as the separator.

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Topic covered: Conditional Statements (if, if…else, nested if, elif, pass and Strings,),

Learning Objective: Learner would be able to

1. Analyze the scenario to write script applying conditional statements.
2. Infer the features of conditions for true and false decision-making.
3. Solve the problems using decision-making statements
4. Process string and perform various slicing operations on string

Theory:

- **The if statement:**

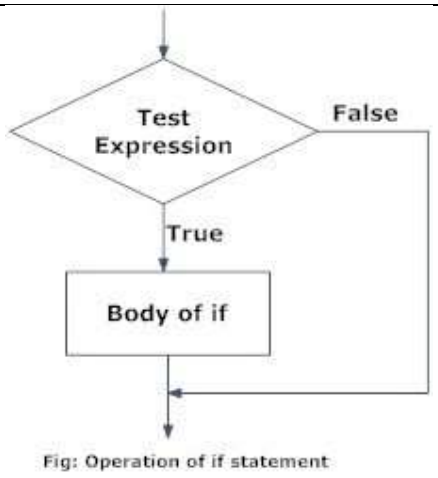
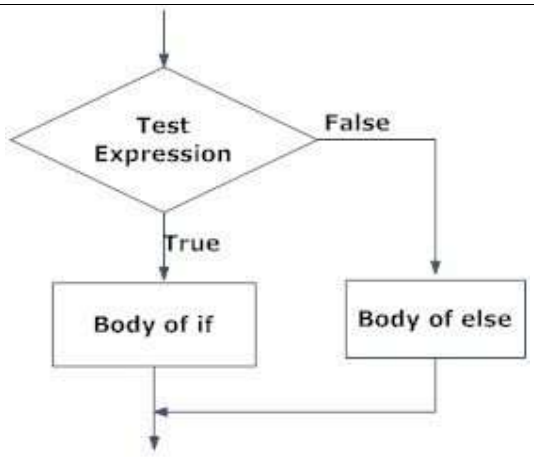
Syntax of if statement	
if test_expression: **body of if**	
The program evaluates the test expression and will execute statement(s) only if the test expression is True. If the test expression is False, the statement(s) is not executed. In Python, the body of the if statement is indicated by the indentation. The body starts with an indentation and the first unindented line marks the end. Python interprets non-zero values as True. None and 0 are interpreted as False.	

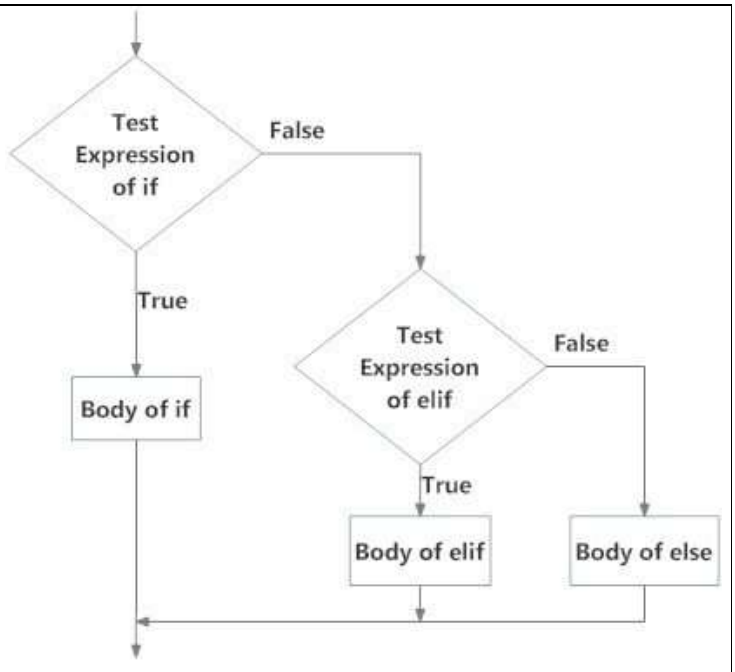
Fig: Operation of if statement

- **The if…else statement:**

Syntax of if…else statement	
if test_expression: **body of if**	
The if..else statement evaluates test expression and will execute the body of if only when the test condition is True. If the condition is False, the body of else is executed. Indentation is used to separate the blocks.	Fig: Operation of if...else statement

- **The if…elif…else statement:**

  The if…elif…else statement is used for decision making.

Syntax of for Loop	
if test_expression: **body of if** **elif test_expression:** **body of elif** **else:** **body of else**	
The elif is short for else if. It allows us to check for multiple expressions. If the condition for if is False, it checks the condition of the next elif block and so on.	Fig: Operation of if...elif...else statement

If all the conditions are False, the body of else is executed. Only one block among the several if...elif...else block s is executed according to the condition. The if block can have only one else block. But it can have multiple elif blocks.	

```
# Check if number is positive or negative or zero:
num = int(input("Enter a number "))
if num > 0:
   print("Positive number")
elif num == 0:
   print("Zero")
else:
   print("Negative number")
```

Output:

Enter a number 2

Positive number

- **The Nested if statement:**

  We can have a if...elif...else statement inside another if...elif...else statement. This is called nesting in computer programming.

  Any number of these statements can be nested inside one another. Indentation is the only way to figure out the level of nesting.

```
# Check if number is positive or negative or zero:
num = int(input("Enter a number: "))
if num >= 0:
   if num == 0:
      print("Zero")
   else:
      print("Positive number")
else:
   print("Negative number")
```

Output:
Enter a number: 0
Zero

String

- String literals in python are surrounded by either single quotation marks, or double quotation marks.
- 'hello' is the same as "hello".
- You can display a string literal with the print( ) function:
- Example

```
print("Hello")
print('Hello')
```

- Assigning a string to a variable is done with the variable name followed by an equal sign and the string:
- Example

```
a = "Hello"
print(a)
```

Str --- String

```
name=u'\u092a\u0948\u0925\u0964\u0928'
print(name)
```

 # Slicing operations on string [start:stop:stepsize], if start and stop is not specified it takes 0 and n-1 and if step size in not specified it is 1.

 # slicing the string stringname[start:stop:stepsize]

```
str='core python'
print(str[0:9:1])
```

```
#str[0:9:2]  --> stepsize 2
#str[2:4:1]  -->  2 to 3 in size 1
#str[ : :2]  -->  access entire string in steps of 2.
#str[ :4: ]  -->  access string from 0 to 3 and stepsize is 1.
#str[-4:-1: ]-->  access string from -4 to -2 from left to right
#str[-1:-4:-1]->  -1 to -3 from right to left...
#str[-1::-1] -->  retrieves from last to first element [reverse]
```

#Repeating String...

```
print(4*"Python")
```

Concatenation (Joining)

```
s1="Core"
s2="Python"
s=s1+s2
print(s)
```

#Checking Membership

Write a program to find sub string in main string...

```
#operator:- in -- if found returns true  ---And--- not in-- returns false if not found
str=input('Enter String')
sub=input('Enter sub string')
if sub in str:
   print("Found")
else:
   print("Not Found")
```

 # Comparing String...

```
s1='Box'
s2="box"
if(s1==s2):
   print("Same")
else:
   print("Not Same")
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 2

Part B (to be completed by students)

Conditional Statements and Strings

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp1)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp1.pdf).
6. Upload the pdf on the web portal

```python
# accept 3 sides of triangle and tell its type
a = float(input("Enter 1st Side of the triangle : "))
b = float(input("Enter 2nd Side of the triangle : "))
c = float(input("Enter 3rd Side of the triangle : "))

if(a==b and b==c):
    print("Equilateral Triangle")
elif(a==b or b==c or a==c):
    print("Isoceles Triangle")
else:
    print("Scalene Triangle")
```

```
Enter 1st Side of the triangle : 1
Enter 2nd Side of the triangle : 1
Enter 3rd Side of the triangle : 1
Equilateral Triangle
```

```python
#function solver

print("f(x) = ax^3 - bx^2 + cx -d            if x > k")
print("f(x) = 0                             if x = k")
print("f(x) = -ax^3 + bx^2 - cx +d          if x < k")

a=float(input("Enter value of a : "))
b=float(input("Enter value of b : "))
c=float(input("Enter value of c : "))
d=float(input("Enter value of d : "))
k=float(input("Enter value of k : "))
x=float(input("Enter value of x : "))

if(x>k):
    f = (a*(x**3))-(b*(x**2))+(c*x)-d
    print(f"Solution : {f}")
elif(x==k):
    print("Solution : 0")
else:
    f = (a*(x**3))-(b*(x**2))+(c*x)-d
    print(f"Solution : {f}")
```

```
f(x) = ax^3 - bx^2 + cx -d            if x > k
f(x) = 0                              if x = k
f(x) = -ax^3 + bx^2 - cx +d           if x < k
Enter value of a : 1
Enter value of b : 2
Enter value of c : 3
Enter value of d : 4
Enter value of k : 5
Enter value of x : 6
Solution : 158.0
```

```python
# BMI calculator
w = float(input("Enter weight in kg : "))
h = float(input("Enter height in m : "))

bmi = w/(h**2)

if(bmi <= 18.4):
    print(f"BMI : {bmi:.2f} Status : Underweight")
elif(bmi <= 24.9):
    print(f"BMI : {bmi:.2f} Status : Normal")
elif(bmi <= 39.9):
    print(f"BMI : {bmi:.2f} Status : Overweight")
else:
    print(f"BMI : {bmi:.2f} Status : Obese")
```

```
Enter weight in kg : 105
Enter height in m : 1.83
BMI : 31.35 Status : Overweight
```

```python
#DISTANCE CONVERTER

print("Enter the value of the convereter u want to use")
print("1. Inches to Feet ")
print("2. Centimeter to Meter ")

choice = int(input("Enter the value : "))

if (choice == 1):
    inch = float(input("Enter distance in inches : "))
    feet = inch/12
    print(f"feet : {feet:.2f}")
elif(choice == 2):
    cm = float(input("Enter distance in centimeter : "))
    m = cm/100
    print(f"meter : {m:.2f}m")
else:
    print("Invalid Choice")
```

```
Enter the value of the convereter u want to use
1. Inches to Feet
2. Centimeter to Meter
Enter the value : 2
Enter distance in centimeter : 183
meter : 1.83m
```

```python
#read and display string
s = input("Enter String : ")
print(s)
```

```
Enter String : Hello World1
Hello World1
```

```python
# length of string
s = "Hello World"
print(len(s))
```

```
11
```

```python
# lower case
s = "Python Programming"
s=s.lower()
print(s)
```

```
python programming
```

```python
# concat
s1="Data"
s2="Science"
s = s1 + " " + s2
print(s)
```

```
Data Science
```

```python
#remove trailing spaces
s = "    Data Analysis    "
print(s)
s = s.strip()
print(s)
```

```
    data analysis
data analysis
```

```python
#odd string elements
s ="Data Science"
print(s[1::2])
```

```
aaSine
```

```python
#check if palindrom or not
s = input("Enter String : ")
rs = s[::-1]
if (s == rs ):
    print("Palindrome")
else :
    print("Not Palindrome")
```

```
Enter String : amma
Palindrome
```

```python
#string reverse
s="Hello"
print(s[::-1])
```

```
olleH
```

```python
# the seperator and joiner
s = "Hello world! I am Tired saying this same word"
a = s.split()
sym = input("enter joiner : ")
print(s)
print(a)
print(sym.join(a))
```

```
enter joiner : -
Hello world! I am Tired saying this same word
['Hello', 'world!', 'I', 'am', 'Tired', 'saying', 'this', 'same',
'word']
Hello-world!-I-am-Tired-saying-this-same-word
```

```python
# input words then reverse words order then join by hypen
s = input("Enter the words : ")
a = s.split()
a = a[::-1]
a = '-'.join(a)
print(a)
```

```
Enter the words : Hello world! I am Tired saying this same word
word-same-this-saying-Tired-am-I-world!-Hello
```

Name :- Aaryan N. Raorane Roll no :- 5004 branch :- AI & DS
batch :- A1 SAPID :- 60018250070

Experiment No 2

Title :- Decision - making and control statements

Aim :- To Implement python programs to demonstrate different decision making and control statements.

Theory :-

1. What is the difference between if, if.... else, and if... elf in Python? Give Examples.

⇒ **if statement :-**

The if statement is the simplest form of decision making in python. The block of code inside it executes only when the given condition evaluates to True. If the condition is False, the block is skipped Entirely and the program moves on.

code :-
```
marks = 75
if marks >= 40:
    print("student has passed")

# output : student has passed.
```

if.... else statement :-

The if.... else construct provides exactly two execution paths. If the condition is True, the if block runs. It is

Sundaram
FOR EDUCATIONAL USE

false, the else block runs. One of the two block is guaranteed to execute.

code:
```
marks = 30

if marks >= 40:
    print("Student has passed")
else:
    print("student has failed")

# output: student has failed.
```

if.... elif..... else statement

when there are more than two possible condition to check, the elif (short for 'else if') clause is used. Python evaluates each condition from top to bottom and executes the first block whose condition is True all remaining conditions are skipped. The else block at the end acts as a default case when no condition above it is satisfied.

code:
```
marks = 82
if marks >= 90:
    print("grade: O")
elif marks >= 75:
    print("grade: A")
elif marks >= 60:
    print("grade: B")
```

Sundaram
FOR EDUCATIONAL USE

```
elif marks >= 40:
    print("Grade: C")
else:
    print("Grade: F")

# output: Grade: A
```

The Key difference among the three is the number of conditions handled. if handles one condition, if...else handles two mutually exclusive outcomes, and if.... elif.... else handles multiple conditions in a single, clean structure.

2. Explain nested if statements with a real-life example.

=> A nested if statement is an if block placed inside another if block. Python first evaluates the outer condition. only if the outer condition is true does it enter that block and then evaluate the inner condition. This allows multiple levels of decision-making to be chained together.

Syntax:
```
if condition1:
    if condition2:
        # executes only when both condition1 and 2 are
          True
    else:
        # executes when condition1 is true but condition2
          is False
else:
    # Executes when condition1 is false
```

Sundaram
FOR EDUCATIONAL USE

Example code:

"""A bank approves a loan only if the applicant is above 18 years of age AND has a credit score above 700. If the applicant is underage, the loan is rejected outright without even checking the credit score"""

```
age = 22
credit_score = 750

if age >= 18:
    if credit_score >= 700:
        print("Loan approved.")
    else:
        print("Loan rejected: Low credit score.")
else:
    print("Loan rejected: Applicant is underage")
```

If age = 15, the output would be "Loan rejected: Applicant is underage" without the inner condition ever being checked. This is exactly how nested conditionals behave; the inner block is entirely unreachable when the outer condition fails.

Sundaram®
FOR EDUCATIONAL USE

3. How does Python determine equality between two strings?

→ Python compares two strings using the == operator, which performs a character-by-character comparison based on Unicode code points. Two strings are considered equal if and only if they have the same length and every character at each corresponding index is identical. The comparison is case-sensitive, meaning "Python" and "python" are not equal.

Code:
```
s1 = "Python"
s2 = "Python"
s3 = "python"

print(s1==s2)  #output: True
print(s1==s3)  #output: False.
```

The == operator checks value equality (same content), while is checks identity (whether both variables point to the same object in memory). Two string variables can hold identical content but still fail an is comparison if they are stored at different memory locations.

Code:
```
a = "Hello"
b = a
c = "Hello"
print(a==b)  # True
print(a is b)  # True
print(a is c)  # False
```

Sundaram® FOR EDUCATIONAL USE

4. Why is Indentation important in Python conditional statements? What happens if it is incorrect?

→ Unlike languages such as C, C++, and Java which use curly braces {} to define code blocks, Python uses indentation (whitespaces at the beginning of a line) to indicate which statements belong to a particular block. The Python interpreter treats indentation as part of the language syntax, not just a formatting convention.

All statements inside the same block must be indented by the same ~~block must~~ number of spaces. The standard convention is 4 spaces per indentation level.

Code:

```
x = 10
if x > 5:
    print("x is greater than 5")
    print("This also belongs to the if block")
print("This is outside the if block")

# output:
# x is greater than 5
# This also belongs to the if block
# This is outside the if block
```

If Indentation is missing or inconsistent, Python raises an IndentationError at parse time and the program does not run at all

Sundaram
FOR EDUCATIONAL USE

code: if a>5:
print("missing indentation")
# IndentationError: expected an indented block

d More dangerous situation is logically incorrect indentation, where the code runs without error but produces wrong output. This is harder to catch

code: a = 3

if a>5:
print("a is greater than 5")
print("this should be inside but is not")

The second print runs regardless of the condition.
output: This should be inside but is not.

In this case, the second print was intended to be part of the if block but was placed at the outer level by mistake, causing it to execute unconditionally. No error is raised, making such bugs difficult to detect.

Sundaram
FOR EDUCATIONAL USE

5. Differentiate between string concatenation and repetition in Python with examples.

→ String concatenation :-

Concatenation is the process of joining two or more strings end to end into a single string. In Python, this is done using the + operator. Both operands must be of type str; concatenating a string with an integer directly raises a TypeError.

code :-
```
first = "Data"
second = "Science"

result = first + " " + second
print (result) # output: Data Science
print(type(result)) # output: <class 'str'>

Date = "30th" + " " + "July"
print (Date) # output: 30th July.
```

If one operand is not a string, it must be explicitly converted using str() before concatenation

code :-
```
course = "Semester :"
num = 3
print(course + str(num)) # output: Semester: 3
```

Sundaram®
FOR EDUCATIONAL USE

String Repetition :-

Repetition produces a new string by repeating the original string a specified number of times. In Python, this is done using the * operator between a string and an integer

code :-
```
word = "Ha"
print(word * 3)  # output: HaHaHa

separator = "=" * 20
print(separator) #output: ====================

line="Python!"
print(line * 2)  #output: Python! Python!
```

Key difference :-

Concatenation (+) joins distinct strings together, while repetition (*) multiplies a single string. Concatenation requires atleast two string operands, whereas repetition requires one string and one integer operand. Mixing up the two operations or their types will result in a TypeError at runtime.

Sundaram®
FOR EDUCATIONAL USE

Conclusion :-

Through this experiment, the decision-making constructs in Python namely if, if---else, if---elif--else and nested if statements were studied and implemented with examples. The role of indentation as a syntactic rule in Python was understood along with the errors caused by incorrect indentation. String equality using the == operator and the difference between string concatenation and repetition were also demonstrated practically.

Sundaram
FOR EDUCATIONAL USE

Control Statements Loop

```
#range
range(0,10)
```

```
range(0, 10)
```

```
a=list(range(10))
print(a)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
a=list(range(0,10))
print(a)
a=list(range(1,10,2))
print(a)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[1, 3, 5, 7, 9]
```

```
print(range(0,20))
print(list(range(0,20)))
print(list(range(0,20,3)))
```

```
range(0, 20)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
[0, 3, 6, 9, 12, 15, 18]
```

for loop

```
#syntax : for Val in sequence :
numbers = [6,5,3,8,4,2,5,4,11]
sum=0
for val in numbers:
    sum = sum + val
print("The sum is ",sum)
```

```
The sum is  48
```

for loop with else

```
list_of_digit = [0,1,2,3,4,5,6]
input_digit = int(input("Enter a digit : "))
for i in list_of_digit :
    if (i == input_digit ):
        print("Digit is in the list")
```

```
        break
else :
        print("Digit not found")
```

```
Enter a digit : 55
Digit not found
```

```
list_of_digit = [0,1,2,3,4,5,6]
input_digit = int(input("Enter a digit : "))
for i in list_of_digit :
    if (i == input_digit ):
        print("Digit is in the list")
else :
        print("Digit not found")
```

```
Enter a digit : 2
Digit is in the list
Digit not found
```

```
sum = 0
for i in range(11):
    sum += i
print(sum)
```

```
55
```

```
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(n2):
        print("*",end=" ")
    print()
```

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

```
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(n2):
        print("* ",end=" ")
    print()
```

```
*  *  *  *  *
*  *  *  *  *
*  *  *  *  *
*  *  *  *  *
*  *  *  *  *
```

```python
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(i,n1):
        print("*",end=" ")
    print()
```

```
* * * * *
* * * *
* * *
* *
*
```

```python
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(i):
        print("*",end=" ")
    print()
```

```

*
* *
* * *
* * * *
```

```python
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(i+1):
        print("*",end=" ")
    print()
```

```
*
* *
* * *
* * * *
* * * * *
```

```python
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(i+1):
        print("*",end="")
    print()
```

```
*
**
***
****
*****
```

```
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(i,n1):
        print(" ",end="")
    for k in range(i+1):
        print("*",end="")
    print()
```

```
     *
    **
   ***
  ****
 *****
```

```
n1 = 5
n2 = 5
for i in range(n1):
    for j in range(i+1):
        print("*",end="")
    print()

for i in range(n1):
    for j in range(i,n1):
        print("*",end="")
    print()
for i in range(n1):
    for j in range(i,n1):
        print(" ",end="")
    for k in range(i+1):
        print("*",end="")
    print()
```

```
*
**
***
****
*****
*****
****
***
**
*
     *
    **
   ***
  ****
 *****
```

While Loop

```
n = int(input("Enter n : "))
i=1
sum = 0
while i<=n:
    sum += i
    i +=1
print(sum)
```

```
Enter n : 11
66
```

```
counter = 0
while counter < 3:
    print("Inside Loop")
    counter += 1
else :
    print("Inside Else")
```

```
Inside Loop
Inside Loop
Inside Loop
Inside Else
```

```
n = int(input("Enter a number : "))
i = 2
count = 0

while (i < n):
    if (n % i == 0):
        count += 1
        break  # Move break inside the if-condition to stop
immediately if a divisor is found
    i += 1     # Increment i at the while-loop level, not inside the
if-statement

if (count == 0 and n > 1):  # A prime number has 0 divisors between 2
and n-1, and must be > 1
    print("Prime")
else:
    print("Not prime")
```

```
Enter a number : 7
Prime
```

```
for i in range (1,51):
    if(i%2!=0):
        continue
    if(i%2==0):
        print(i,end=" ")
```

```
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50
```

```
c=0
while c<10:
    if c==3:
        print("using continue stateemnt : skipping iteration when
count is : ",c)
        c=c+1
        continue
    if c==6:
        print("using pass statement : count is : ",c)
        c=c+1
        pass
    if c==8:
        print("using break statement : count is : ",c)
        c=c+1
        break
    print("Current count is : ",c)
    c=c+1
```

```
Current count is :  0
Current count is :  1
Current count is :  2
using continue stateemnt : skipping iteration when count is :  3
Current count is :  4
Current count is :  5
using pass statement : count is :  6
Current count is :  7
using break statement : count is :  8
```

```
for i in range(1,21,2):
    print(i)
```

```
1
3
5
7
9
11
13
15
17
19
```

```
for i in range(6):
    print(i)
```

```
0
1
2
3
```

```
4
5

# 1 to 10
for i in range(1,11):
    print(i,end=" ")

1 2 3 4 5 6 7 8 9 10

# 10 to 1
for i in range(10,0,-1):
    print(i,end=" ")

10 9 8 7 6 5 4 3 2 1

# table of
n = int(input("Enter n: "))
for i in range(1,11):
    print(f"{n} X {i} = {n*i}")

Enter n: 5
5 X 1 = 5
5 X 2 = 10
5 X 3 = 15
5 X 4 = 20
5 X 5 = 25
5 X 6 = 30
5 X 7 = 35
5 X 8 = 40
5 X 9 = 45
5 X 10 = 50

# factorial of a num
n = int(input("Enter n : "))
fact = 1
for i in range(1,n+1):
    fact = fact * i
print(fact)

Enter n : 5
120

# sum of digits
n = int(input("Enter n : "))
sum =0
while(n!=0) :
    sum = sum + int(n%10)
    n=int(n/10)
print(sum)
```

```
Enter n : 123
6

# Reverse of a number
n = int(input("Enter n : "))
rn = 0

while n != 0:
    rn = (rn * 10) + (n % 10)
    n = n // 10

print(rn)

Enter n : 445
544

#palindrome
# Reverse of a number
n = int(input("Enter n : "))
n1 = n
rn = 0

while n != 0:
    rn = (rn * 10) + (n % 10)
    n = n // 10  # Integer division to reduce n

if (rn == n1):
    print("palindrome")
else :
    print("not palindrome")

Enter n : 252
palindrome

# fibonacci
a = 0
b = 1
n = int(input("enter no of elements in series u want : "))
for i in range (n):
    print(f"{a}",end=" ")
    a,b = b,a+b

enter no of elements in series u want : 6
0 1 1 2 3 5

#traverse string

s = input("Enter a string")

for i in range(len(s)):
    print(f"{i} -> {s[i]}")
```

```
Enter a stringraphael
0 -> r
1 -> a
2 -> p
3 -> h
4 -> a
5 -> e
6 -> l
```

```
#pattern 1
for i in range(1,6):
    for k in range(6-i):
        print(" ",end=" ")
    for j in range(i):
        print("*",end=" ")
    print()
```

```
          *
        * *
      * * *
    * * * *
  * * * * *
```

```
#pattern 2
for i in range(5):
    for j in range(1,6):
        print(j,end="")
    print()
```

```
12345
12345
12345
12345
12345
```

```
#pattern 3
for i in range(1,6):
    for j in range(1,i+1):
        print(j,end="")
    print()
```

```
1
12
123
1234
12345
```

```
#patttern 4
k=65
for i in range(1,6):
    for j in range(1,i+1):
```

```
        print(chr(k),end="")
        k+=1
    print()
```

```
A
BC
DEF
GHIJ
KLMNO
```

```
#patttern 5
k=65
for i in range(1,6):
    for j in range(1,i+1):
        print(chr(k),end="")
    k+=1
    print()
```

```
A
BB
CCC
DDDD
EEEEE
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 3

Part A (To be referred by students)

Looping & Strings

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp3)

Aim: Write a program to demonstrate Looping Statements (while, for, nested loops, break, continue, else with loop), Pass and string functions.

Problem Statement: Write Python program to

1. WAP to print the sum of the first n natural numbers using a for loop
2. Count the number of even and odd numbers from a series of numbers P to Q using a for loop.
3. Write a program to check whether the entered number is prime or not (make use of break)
4. Display the Fibonacci sequence up to the nth term, where n is provided by the user (use a while loop).
5. Print the following pattern using a nested loop.

```
  *
 ***
*****
```

6. WAP to print the following pattern

```
*****        1          *
****         12         **
***          123        ***
**           1234       **
*            12345      *
```

7. Write a Python program that iterates the integers from 1 to 50. For multiples of three print "Fizz" instead of the number and for multiples of five print "Buzz". For numbers that are multiples of three and five, print "FizzBuzz".
   Sample Output :
   fizzbuzz
   1
   2
   fizz
   4
   buzz
8. Demonstrate 5 string operations using string function
9. Demonstrate using the continue statement in a loop
10. Demonstrate use of Pass in loop and else with loop

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Topic covered: Loops (for, while) else with loop, pass & string functions

Learning Objective: Learner would be able to

1. Analyze the scenario to write script applying looping.
2. Solve the problems using looping statements
3. Understand and use pass, else, continue & break
4. Process string and perform various operations on string

Theory:

- **The for loop:**

  The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects. With for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

Syntax of for Loop

```
for val in sequence:
    loop body
```

Here, val is the variable that takes the value of the item inside the sequence on each iteration.

Loop continues until we reach the last item in the sequence. The body of for loop is separated from the rest of the code using indentation.

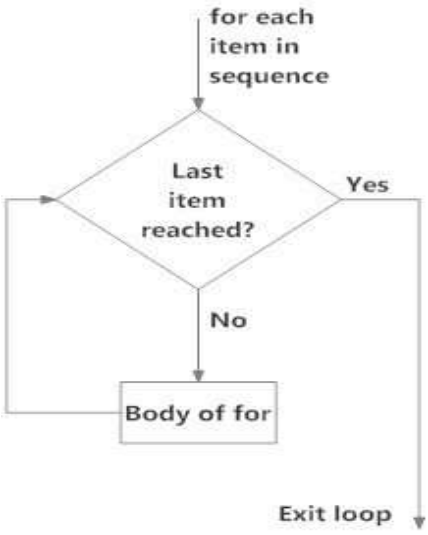


Fig: operation of for loop

The for loop does not require an indexing variable to set beforehand, as the for command itself allows for this.

```
# Print each fruit in a fruit list:
fruits = ["apple", "banana", "cherry"]
for x in fruits:
```

```
print(x)
```

Output:

apple

banana

cherry

- **The while loop:**

  The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects. With for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

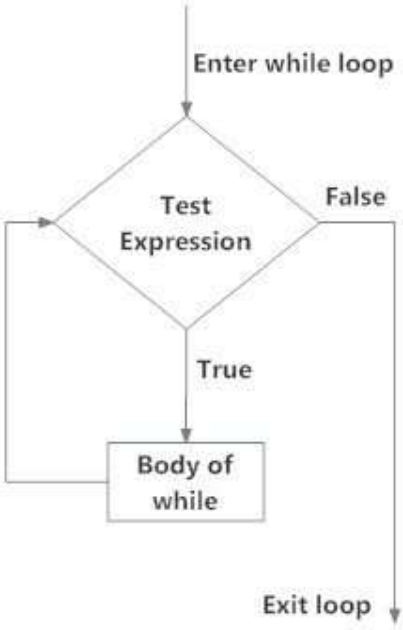
Syntax of while Loop	
while test_expression: **loop body**	
In the while loop, test expression is checked first. The body of the loop is entered only if the test_expression evaluates to True. The body starts with indentation and the first unindented line marks the end. Python interprets any non-zero value as True. None and 0 are interpreted as False.	

Fig: operation of while loop

```
# Program to add natural numbers up to sum = 1+2+3+...+n
n = 10
# initialize sum and counter
sum = 0
i = 1
while i <= n:
    sum = sum + i
    i = i+1    # update counter
# print the sum
print("The sum is", sum)
```

Output:

```
The sum is 55
```

- **The range() function:**

  To loop through a set of code a specified number of times, we can use the range() function, The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

  We can also define the start, stop and step size as range(start, stop,step_size). step_size defaults to 1 if not provided.

```
# range(n) prints numbers 0 to n-1
# Outputs 0 to 5
for x in range(6):
    print(x)
# Outputs 2 to 5
for x in range(2, 6):
    print(x)
# Outputs 2, 5, 8, 11, 14, 17
for x in range(2, 20, 3):
    print(x)
```

- **The break Statement:**

  The break statement terminates the loop containing it. Control of the program flows to the statement immediately after the body of the loop.

  If the break statement is inside a nested loop (loop inside another loop), the break statement will terminate the innermost loop.

```
# Exit the loop when x is banana
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        break
    print(x)
```

Output:

apple

- **The continue Statement:**

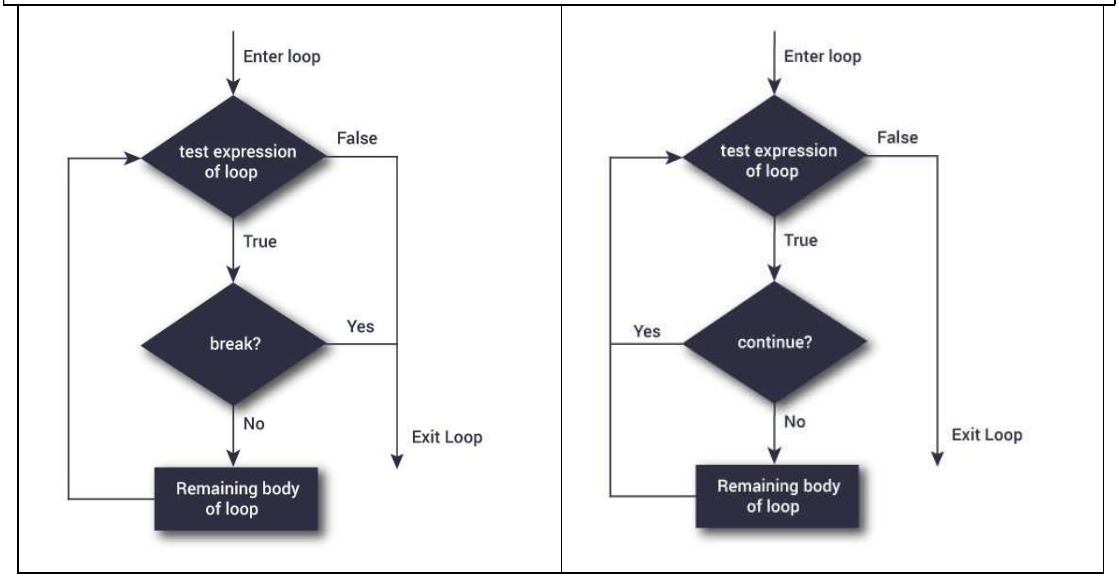
  The continue statement is used to skip the rest of the code inside a loop for the current iteration only. Loop does not terminate but continues on with the next iteration.

```
# Exit the loop when x is banana
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        continue
    print(x)
```

Output:

apple

cherry



- **Loop with else (use with for and while loop):**

A for loop can have an optional else block as well. The else part is executed if the items in the sequence used in for loop exhausts.

The break keyword can be used to stop a for loop. In such cases, the else part is ignored. Hence, a for loop's else part runs if no break occurs.

```
digits = [0, 1, 5]
for i in digits:
    print(i)
else:
    print("No items left.")
```

Output:

0

1

5

No items left.

Strings….

- **Python Strings are immutable**

  In Python, strings are immutable. That means the characters of a string cannot be changed.

```
message = 'Hola Amigos'
message[0] = 'H'
print(message)
```

Output

TypeError: 'str' object does not support item assignment

However, we can assign the variable name to a new string.

```
message = 'Hola Amigos'
# assign new string to message variable
message = 'Hello Friends'
prints(message); # prints "Hello Friends"
```

- **Python Multiline String**

  We can also create a multiline string in Python. For this, we use triple double quotes """ or triple single quotes '''.

```
# multiline string
message = """
Never gonna give you up
Never gonna let you down
"""
print(message)
```

Output

```
Never gonna give you up
Never gonna let you down
```

- **String Operations:**

  There are many operations that can be performed with strings which makes it one of the most used data types in Python.

  1. Compare Two Strings

  We use the == operator to compare two strings. If two strings are equal, the operator returns True. Otherwise, it returns False.

```
str1 = "Hello, world!"
```

```
str2 = "I love Python."
str3 = "Hello, world!"
# compare str1 and str2
print(str1 == str2)
# compare str1 and str3
print(str1 == str3)
```

Output

```
False
True
```

In the above example,

str1 and str2 are not equal. Hence, the result is False.

str1 and str3 are equal. Hence, the result is True.

2. Join Two or More Strings

In Python, we can join (concatenate) two or more strings using the + operator.

```
greet = "Hello, "
name = "Jack"

# using + operator
result = greet + name
print(result)
```

Output

```
Hello, Jack
```

Run Code

3. Iterate Through a Python String

We can iterate through a string using a for loop.

```
greet = 'Hello'
# iterating through greet string
for letter in greet:
    print(letter)
```

Output

```
H
e
l
l
o
```

5. Python String Length

In Python, we use the len() method to find the length of a string.

```
greet = 'Hello'
# count length of greet string
print(len(greet))
```

Output

```
5
```

6. String Membership Test

We can test if a substring exists within a string or not, using the keyword in.

```
print('a' in 'program') # True
print('at' not in 'battle') False
```

7. Methods of Python String

Besides those mentioned above, there are various string methods present in Python. Here are some of those methods:

Methods	Description
upper()	converts the string to uppercase
lower()	converts the string to lowercase
partition()	returns a tuple
replace()	replaces substring inside
find()	returns the index of first occurrence of substring
rstrip()	removes trailing characters
split()	splits string from left
startswith()	checks if string starts with the specified string
isnumeric()	checks numeric characters
index()	returns index of substring

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 3

Part B (to be completed by students)

Looping and Strigs

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp3)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp3.pdf).
6. Upload the pdf on the web portal

```python
#sum of 1st n natural numbers
sum = 0
n=int(input("Enter no of elements : "))
for i in range(1,n+1):
    sum+=i
print(sum)
```

```
Enter no of elements : 10
55
```

```python
#count number of even and odd numbers in the given series
p=int(input("enter p :"))
q=int(input("enter q :"))
even = 0
odd = 0
for i in range(p,q+1):
    if(i%2 == 0):
        even+=1
    else:
        odd+=1

print(f"Total Even numbers : {even}")
print(f"Total Odd numbers : {odd}")
```

```
enter p :51
enter q :60
Total Even numbers : 5
Total Odd numbers : 5
```

```python
#prime checker
n = int(input("Enter a number : "))
i = 2
count = 0

while (i < n):
    if (n % i == 0):
        count += 1
        break
    i += 1

if (count == 0 and n > 1):
    print("Prime")
else:
    print("Not prime")
```

```
Enter no : 4
```

```python
#fibonnaci
n = int(input("Enter the last term u want in fibonacci series : "))
a=0
b=1
```

```
while(a <= n):
    print(a,end=" ")
    a,b=b,a+b
```

```
Enter the last term u want in fibonacci series : 13
0 1 1 2 3 5 8 13
```

```
#nested pattern loop
for i in range(1,6):
    for j in range(6-i):
        print(" ",end="")
    for k in range(2*i-1):
        print("*",end="")
    print()
```

```
     *
    ***
   *****
  *******
 *********
```

```
#pattern 1
for i in range(1,6):
    for j in range (6-i):
        print("*",end=" ")
    print()
```

```
* * * * *
* * * *
* * *
* *
*
```

```
#pattern 2
for i in range(1,6):
    for j in range(1,i+1):
        print(j,end="")
    print()
```

```
1
12
123
1234
12345
```

```
#pattern 3
n1 = 3
for i in range(n1):
    for j in range(i+1):
        print("* ",end="")
    print()
```

```python
for i in range(n1):
    for j in range(i,n1-1):
        print("* ",end="")
    print()
```

```
*
* *
* * *
* *
*
```

```python
#fizzbuzz
for i in range(1,51):
    if(i%3==0 and i%5!=0):
        print("Fizz")
    elif(i%3!=0 and i%5==0):
        print("Buzz")
    elif(i%3==0 and i%5==0):
        print("FizzBuzz")
    else:
        print(i)
```

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
Fizz
22
23
Fizz
Buzz
26
```

```
Fizz
28
29
FizzBuzz
31
32
Fizz
34
Buzz
Fizz
37
38
Fizz
Buzz
41
Fizz
43
44
FizzBuzz
46
47
Fizz
49
Buzz
```

```
#string operations
s = input("enter string sentence: ")
print(s.upper())
print(s.lower())
print(s.capitalize())
s=s.split()
print(s)
s='_@_'.join(s)
print(s)
```

```
enter string sentence: Hello world i am python not language snake
HELLO WORLD I AM PYTHON NOT LANGUAGE SNAKE
hello world i am python not language snake
Hello world i am python not language snake
['Hello', 'world', 'i', 'am', 'python', 'not', 'language', 'snake']
Hello_@_world_@_i_@_am_@_python_@_not_@_language_@_snake
```

```
#continue
for i in range(10):
    if(i%2==0):
        continue
    print(i)
```

```
1
3
```

```
5
7
9

#pass and else
for i in range(11):
    if(i==3):
        pass
    print(i)
else:
    print("Loop complete")

0
1
2
3
4
5
6
7
8
9
10
Loop complete
```

Name:- Aaryan NRaorane Roll No:- 5004 Batch:- A1
branch:- AI&DS SAP ID:- 60018250070

Experiment No 3

Title: Looping and strings

Aim: Write a program to demonstrate looping statements (while, for, nested loops, break, continue, else with loop), lists and string functions

Theory:

1) Differentiate between for loop and while loop with examples.

⇒ The for loop is used when the number of iterations is known in advance. It iterates over a sequence such as a list, string, or range object, and the loop variable takes each value from that sequence automatically. The while loop is used when the number of iterations depends on a condition. The loop body executes as long as the condition remains true, and the programmer must manually update the condition variable to avoid infinite execution.

Parameter	for loop	while loop
• Iteration control	over a sequence	Based on a condition
• Number of iterations	Known in advance	May be unknown
• counter update	Automatic	Manual
• Risk of infinite loop	Lower	Higher if condition not updated.

Sundaram
FOR EDUCATIONAL USE

Example of for loop:

```
code: for i in range(1, 6):
          print("Iteration:", i)
```

output:
```
Iteration : 1
Iteration : 2
Iteration : 3
Iteration : 4
Iteration : 5
```

Example of while loop:

```
code: i = 1
      while i <= 5
          print("Iteration:", i)
          i += 1
```

output:
```
Iteration : 1
Iteration : 2
Iteration : 3
Iteration : 4
Iteration : 5
```

Sundaram
FOR EDUCATIONAL USE

2) Explain the role of the range() function in loops. Give two examples.

=> The range() function generates a sequence of integers and is commonly used with for loops to control the number of iterations. It accepts up to three arguments: start, stop, and step. The start value defaults to 0 and the step defaults to 1. The stop value is exclusive, meaning the sequence ends one integer before it.

Syntax:

```
range(stop)
range(start, stop)
range(start, stop, step)
```

Example 1:

~~for code:~~

```
for i in range(1,6)
    print("Iteration:", i)
```

output:

```
Iteration : 1
Iteration : 2
Iteration : 3
Iteration : 4
Iteration : 5
```

Sundaram®
FOR EDUCATIONAL USE

Example 2: ~~Print~~

code:- for i in range(1,10,2)
print(i)

output:- 1 3 5 7 9

3) what is the purpose of the else block in a loop? Write one example using for loop with else.

=> The else block in a loop executes after the loop completes all its iterations normally. If the loop is terminated early by a break statement, the else block is skipped entirely. This makes it useful for detecting whether a loop ran to completion without interruption, such as in search operations where a break is used when the target is found.

code:- for i in range(1,6):
print(i)
else:
print("Loop completed without break")

output:- 1
2
3
4
5
Loop completed without break.

since no break was encountered, the else block is executed.

Sundaram FOR EDUCATIONAL USE

4) Differentiate between break, continue, and pass with suitable examples.

=>

Keyword	Effect on loop	common use case
break	Exits the loop actually	stop execution when condition is met
continue	skips the current iteration	Bypass specific value in sequence
pass	no effect; loop proceeds normally	Placeholder where syntax requires a statement

Example of break:

```
for i in range(1, 10):
    if i == 5:
        break
    print(i, end=" ")
```

output: 1 2 3 4

The loop exits as soon as i reaches 5

Example of continue:

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i, end=" ")
```

output: 1 2 4 5

The value 3 is skipped, all other iterations proceed normally.

Sundaram
FOR EDUCATIONAL USE

Example of pass:

```
for j in range(1,6):
    if j == 3:
        pass
    print(j, end=" ")
```

output: 1 2 3 4 5

5) Why are strings called immutable in Python? Explain with an example.

=> Strings in Python are called immutable because once a string object is created in memory, its individual characters cannot be modified in place. Any operation that appears to change a string doesn't alter the original object. Instead, Python creates a new string object with the modified content and assigns it to the variable. The original object remains unchanged.

This can be verified using the id() function, which returns the memory address of an object. After reassignment, the address changes, confirming that a new object was created.

code:

```
s = "hello"
print("Before:", id(s))

s = s + "world"
print("After:", id(s))
```

Sundaram
FOR EDUCATIONAL USE

output: Before: 140245678401234
After: 140245 67890 1235

The two different id values confirm that ~~assignment~~ reassignment created a new object rather than modifying the original string.

conclusion:-

This Experiment covered the core looping constructs in Python including for, while, nested loops, and loop control statement such as break, continue and pass. The behaviour of the else block with loops and the use of the range() function were also observed through code execution. String immutability was verified by attempting direct character modification and comparing object identifies using id().

Sundaram®
FOR EDUCATIONAL USE

List,Tuple,Dictionary

1. List

```
thisList = ["apple","banana","Cherry","dragonfruit"]
print(thisList)

thisList = list(("apple","banana","Cherry","dragonfruit"))
print(thisList)

['apple', 'banana', 'Cherry', 'dragonfruit']
```

List allows duplicate,it is changeable and ordered(like array)

```
print(type(thisList))

<class 'list'>
```

1. Len

```
print(len(thisList))

4

list1 = ["abc",34,True,40,'z']
print(list1)
print(type(list1))

['abc', 34, True, 40, 'z']
<class 'list'>
```

The slicing and element callling in list is similar to that of string

```
print("list1[2] = ",list1[2])

list1[2] =  True

print("list1[:2] = ",list1[:2])

list1[:2] =  ['abc', 34]

print("list1[-1] = ",list1[-1])

list1[-1] =  z

fruit = ["apple","banana","cherry","orange","Kiwi","Melon","Mango"]
```

```
print(fruit)
print(fruit[2:5])
print(fruit[:4])
```

```
['apple', 'banana', 'cherry', 'orange', 'Kiwi', 'Melon', 'Mango']
['cherry', 'orange', 'Kiwi']
['apple', 'banana', 'cherry', 'orange']
```

```
print(fruit[-4:-1])
```

```
['orange', 'Kiwi', 'Melon']
```

```
fruit[1]="blackcurrent"

print(fruit)
```

```
['apple', 'blackcurrent', 'cherry', 'orange', 'Kiwi', 'Melon',
'Mango']
```

```
fruit[2:4]=["chickoo","dragonfruit"]

print(fruit)
```

```
['apple', 'blackcurrent', 'chickoo', 'dragonfruit', 'Kiwi', 'Melon',
'Mango']
```

```
fruit.append("pineapple")

print(fruit)
```

```
['apple', 'blackcurrent', 'chickoo', 'dragonfruit', 'Kiwi', 'Melon',
'Mango', 'pineapple']
```

```
fruit.insert(3,"watermelon")

print(fruit)
```

```
['apple', 'blackcurrent', 'chickoo', 'watermelon', 'dragonfruit',
'Kiwi', 'Melon', 'Mango', 'pineapple']
```

```
tropical=["papaya","jackfruit","gauva"]
fruit.extend(tropical)

print(fruit)
```

```
['apple', 'blackcurrent', 'chickoo', 'watermelon', 'dragonfruit',
'Kiwi', 'Melon', 'Mango', 'pineapple', 'papaya', 'jackfruit', 'gauva']
```

```
fruittuple=("lichi","grapes")

fruit.extend(fruittuple)

print(fruit)
```

```
['apple', 'blackcurrent', 'chickoo', 'watermelon', 'dragonfruit',
'Kiwi', 'Melon', 'Mango', 'pineapple', 'papaya', 'jackfruit', 'gauva',
'lichi', 'grapes']

fruit.remove("papaya")
print(fruit)

['apple', 'blackcurrent', 'chickoo', 'watermelon', 'dragonfruit',
'Kiwi', 'Melon', 'Mango', 'pineapple', 'jackfruit', 'gauva', 'lichi',
'grapes']

fruit.pop(2)
print(fruit)

['apple', 'blackcurrent', 'watermelon', 'dragonfruit', 'Kiwi',
'Melon', 'Mango', 'pineapple', 'jackfruit', 'gauva', 'lichi',
'grapes']

del fruit[5]
print(fruit)

['apple', 'blackcurrent', 'watermelon', 'dragonfruit', 'Kiwi',
'Mango', 'pineapple', 'jackfruit', 'gauva', 'lichi', 'grapes']

#fruit.clear()
#print(fruit)

# del fruit
# print(fruit)

for x in fruit:
    print(x)

apple
blackcurrent
watermelon
dragonfruit
Kiwi
Mango
pineapple
jackfruit
gauva
lichi
grapes

print("apple" in fruit)

True

fruit.sort()
print(fruit)
```

```
['Kiwi', 'Mango', 'apple', 'blackcurrent', 'dragonfruit', 'gauva',
'grapes', 'jackfruit', 'lichi', 'pineapple', 'watermelon']
```

```
fruit.sort(reverse=True)
print(fruit)
```

```
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva',
'dragonfruit', 'blackcurrent', 'apple', 'Mango', 'Kiwi']
```

```
!jt -t onedork -T -N
```

```
fruit2 = fruit#updates both list even after altering
print(fruit2)
fruit.append("berry")
print(fruit)
print(fruit2)
```

```
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva',
'dragonfruit', 'blackcurrent', 'apple', 'Mango', 'Kiwi']
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva',
'dragonfruit', 'blackcurrent', 'apple', 'Mango', 'Kiwi', 'berry']
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva',
'dragonfruit', 'blackcurrent', 'apple', 'Mango', 'Kiwi', 'berry']
```

```
fruit2 = fruit.copy()
print(fruit2)#doesnt updates
fruit.append("blueberry")
print(fruit)
print(fruit2)
```

```
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva',
'dragonfruit', 'blackcurrent', 'apple', 'Mango', 'Kiwi', 'berry']
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva',
'dragonfruit', 'blackcurrent', 'apple', 'Mango', 'Kiwi', 'berry',
'blueberry']
['watermelon', 'pineapple', 'lichi', 'jackfruit', 'grapes', 'gauva',
'dragonfruit', 'blackcurrent', 'apple', 'Mango', 'Kiwi', 'berry']
```

```
fruit3 = fruit + fruit2
fruit3.sort()
print(fruit3)
```

```
['Kiwi', 'Kiwi', 'Mango', 'Mango', 'apple', 'apple', 'berry', 'berry',
'blackcurrent', 'blackcurrent', 'blueberry', 'dragonfruit',
'dragonfruit', 'gauva', 'gauva', 'grapes', 'grapes', 'jackfruit',
'jackfruit', 'lichi', 'lichi', 'pineapple', 'pineapple', 'watermelon',
'watermelon']
```

```
for x in fruit2:
    fruit.append(x)
```

```
fruit.sort()
print(fruit)
```

```
['Kiwi', 'Kiwi', 'Mango', 'Mango', 'apple', 'apple', 'berry', 'berry',
'blackcurrent', 'blackcurrent', 'blueberry', 'dragonfruit',
'dragonfruit', 'gauva', 'gauva', 'grapes', 'grapes', 'jackfruit',
'jackfruit', 'lichi', 'lichi', 'pineapple', 'pineapple', 'watermelon',
'watermelon']
```

```
berries = ["blueberry","strawberry","mulberry"]
berries*3
print(berries)
```

```
['blueberry', 'strawberry', 'mulberry']
```

```
sq = [x**2 for x in range(1,6)]
print(sq)
```

```
[1, 4, 9, 16, 25]
```

```
even = [x for x in range(10) if x%2==0]
print(even)
```

```
[0, 2, 4, 6, 8]
```

```
upper_fruit=[w.upper() for w in fruit]
print(upper_fruit)
```

```
['KIWI', 'KIWI', 'MANGO', 'MANGO', 'APPLE', 'APPLE', 'BERRY', 'BERRY',
'BLACKCURRENT', 'BLACKCURRENT', 'BLUEBERRY', 'DRAGONFRUIT',
'DRAGONFRUIT', 'GAUVA', 'GAUVA', 'GRAPES', 'GRAPES', 'JACKFRUIT',
'JACKFRUIT', 'LICHI', 'LICHI', 'PINEAPPLE', 'PINEAPPLE', 'WATERMELON',
'WATERMELON']
```

nested list

```
a = [[11,12,13],[21,22,23],[31,32,33]]
print(a)
```

```
[[11, 12, 13], [21, 22, 23], [31, 32, 33]]
```

Tuple

tuple is immutable

```
tfruit=("apple","banana","chickoo","dragonfruit","grape","gauva","jack
fruit","lemon")
print(tfruit)
print(type(tfruit))
```

```
('apple', 'banana', 'chickoo', 'dragonfruit', 'grape', 'gauva',
'jackfruit', 'lemon')
<class 'tuple'>
```

indexing in tuple is similar to list

```
print(type(tfruit[0]))
print(tfruit[0])

<class 'str'>
apple

print(len(tfruit))

8

print(tfruit[1:2])
print(tfruit[:-1])
print(tfruit[-2:])

('banana',)
('apple', 'banana', 'chickoo', 'dragonfruit', 'grape', 'gauva',
'jackfruit')
('jackfruit', 'lemon')
```

we can update a element in tuple by first convertin it into a list and then do changes and convert back to a tuple

```
x=tfruit
y=list(x)
y[1]="kiwi"
x=tuple(y)
print(x)

('apple', 'kiwi', 'chickoo', 'dragonfruit', 'grape', 'gauva',
'jackfruit', 'lemon')

t=('JP morgan','20','education')
print(t)

('JP morgan', '20', 'education')

(company,empid,profile)=t
print(company)
print(empid)
print(profile)

JP morgan
20
education
```

```
t1=(1,2,3,4)
a,b,c,d=t1
print(a,b,c,d)
```

```
1 2 3 4
```

```
a,b,c=t1
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-52-408f20357701> in <module>
----> 1 a,b,c=t1

ValueError: too many values to unpack (expected 3)
```

```
(a,b,c)=t1[0:3]
```

```
print(a,b,c)
```

```
1 2 3
```

```
# Tuples are Comparable

print((0,1,2) < (5,1,2)) # 0 is less than 5 - True

print((0,1,200) < (0,3,4)) # 0 is equal to 0 --> 1 is less than 3 - True

print(('Jones', 'Sally') < ('Jones', 'Sam')) # Jones is equal to Jones --> Sally < Sam because l < m - True

print(('Jones', 'Sally') < ('Adams', 'Sam')) # Jones is not equal to Adams but is greater than Adams - therefore, False
```

```
True
True
True
False
```

```
# Concatenation, Repetition, Index

t1=(1,2,3)
t2=(3,'Priya',4)
t3=t1+t2
print(t3)
print()
print(t1*3)
print()
print(t2.index('Priya'))
```

```
(1, 2, 3, 3, 'Priya', 4)

(1, 2, 3, 1, 2, 3, 1, 2, 3)

1
```

```
l=(1,2,3,1,2,3,1,2,3)
print(2 in l)
```

```
True
```

```
# Iterating through a tuple

for name in ('John', 'Kate'):
    print("Hello", name)
```

```
Hello John
Hello Kate
```

```
fruits=('apple', 'banana', 'mango', 'grapes', 'pineapple')
for x in fruits:
    print(x)
```

```
apple
banana
mango
grapes
pineapple
```

Tuples do not support these methods: append remove insert reverse sort

Set

```
fruits={'apple', 'banana', 'mango', 'grapes', 'pineapple'}
print(fruits)

a={1,3,5,2,4}
print(a)
print(type(a))

# RANDOM ORDER

{'grapes', 'mango', 'banana', 'pineapple', 'apple'}
{1, 2, 3, 4, 5}
<class 'set'>

thisSet=set(('apple', 'banana', 'mango', 'grapes', 'pineapple'))
print(thisSet)
print(type(thisSet))

{'grapes', 'mango', 'banana', 'pineapple', 'apple'}
<class 'set'>

a={1,2,2,3,3,3,4,4,4,4}
print(a)

{1, 2, 3, 4}

a={1,2,3}
print(a[1]) # 'set' object does not support indexing

---------------------------------------------------------------------
-----
TypeError                                 Traceback (most recent call
last)
<ipython-input-17-61a87b252975> in <module>
      1 a={1,2,3}
----> 2 print(a[1]) # 'set' object does not support indexing

TypeError: 'set' object does not support indexing

fruits={'apple', 'banana', 'mango', 'grapes', 'pineapple'}
for x in fruits:
```

```
    print(x)
print()
print(len(fruits))

# Once a set is created, you cannot change its item but you can add
new items
```

```
grapes
mango
banana
pineapple
apple

5
```

```
fruits={'apple', 'banana', 'mango', 'grapes', 'pineapple'}
fruits.add('orange')
print(fruits)

fruits.update(['cherry', 'blackberry', 'dragonfruit'])
print(fruits)

fruits.discard('strawberry') # if the item does not exist, discard()
will not raise an error
print(fruits)

fruits.remove('strawberry') # if the item does not exist, remove()
will raise an error
print(fruits)
```

```
{'orange', 'grapes', 'mango', 'banana', 'pineapple', 'apple'}
{'orange', 'cherry', 'grapes', 'mango', 'banana', 'dragonfruit',
'blackberry', 'pineapple', 'apple'}
{'orange', 'cherry', 'grapes', 'mango', 'banana', 'dragonfruit',
'blackberry', 'pineapple', 'apple'}

---------------------------------------------------------------------
-----
KeyError                                  Traceback (most recent call
last)
<ipython-input-19-93efbfaa3e7d> in <module>
      9 print(fruits)
     10
---> 11 fruits.remove('strawberry') # if the item does not exist,
remove() will raise an error
     12 print(fruits)

KeyError: 'strawberry'
```

```
fruits={"apple", "banana", "mango", "grapes", "pineapple", "papaya",
"guava", "lemon"}
```

```
print()
fruits.clear()
print(fruits)

del fruits
print(fruits)
```

```
set()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-20-2b95bb775634> in <module>
      6
      7 del fruits
----> 8 print(fruits)

NameError: name 'fruits' is not defined
```

```
fruits={"apple", "banana", "mango", "grapes", "pineapple", "papaya",
"guava", "lemon"}

x=fruits.pop()
print(x)
print(fruits)
```

```
grapes
{'lemon', 'guava', 'mango', 'papaya', 'banana', 'pineapple', 'apple'}
```

```
set1={1,2,3,'b'}
set2={'a', 'b', 'c', 1, 3}
set3=set1.union(set2)
print(set3)

print()

set1.update(set2)
print(set1)

# both union and update will exclude any duplicate values
```

```
{1, 2, 3, 'b', 'c', 'a'}

{1, 2, 3, 'b', 'c', 'a'}
```

```
null_set=set()
print(null_set)

a={1, 2.5, ("pineapple", "pumpkin"), "Jack"}
```

```
print(a)

b=set("abcd")
print(b) # {'b', 'a', 'd', 'c'}

c={"abcd"}
print(c) # {'abcd'}

print(len(a))

print(("pineapple", "pumpkin") in a) # True
print(("pineapple") in a) # False

print()

for x in a:
    print(x)
```

```
set()
{1, 2.5, ('pineapple', 'pumpkin'), 'Jack'}
{'d', 'b', 'c', 'a'}
{'abcd'}
4
True
False

1
2.5
('pineapple', 'pumpkin')
Jack
```

```
a_set=set("abcd")
b_set=set("cdef")
print(a_set & b_set) # OR print(a_set.intersection(b_set))

print()

print(a_set - b_set) # OR print(a_set.difference(b_set)) # {'b', 'a'}
print(b_set - a_set) # OR print(b_set.difference(a_set)) # {'e', 'f'}

print()
print(a_set ^ b_set) #{'f', 'e', 'a', 'b'}
# OR b_set.symmetric_difference(a_set) # {'a', 'b', 'e', 'f'}
```

```
{'d', 'c'}

{'b', 'a'}
{'e', 'f'}

{'f', 'b', 'e', 'a'}
```

```
my_set={'a', 'b', 'c'}
print(my_set)

print()

my_copy=my_set.copy()
print(my_copy)

print()

my_ref_copy=my_set
print(my_ref_copy)

print()

my_set.remove('b')
print(my_set) # {'a', 'c'}
print(my_copy) # {'b', 'a', 'c'}
print(my_ref_copy) # {'a', 'c'}
```

```
{'b', 'c', 'a'}

{'b', 'c', 'a'}

{'b', 'c', 'a'}

{'c', 'a'}
{'b', 'c', 'a'}
{'c', 'a'}
```

```
# frozenset() Function - making set immutable
a=(1,2,3,4,5,6,7,8,9,10)
#converting tuple to frozenset
fnum=frozenset(a)
print("Frozenset Object is: ", fnum)
```

```
Frozenset Object is:  frozenset({1, 2, 3, 4, 5, 6, 7, 8, 9, 10})
```

Dictionary

```
thisDict={
    "Brand": "Mahindra",
    "Model": "BE 6e Batman Edition",
    "Year": 2025
}

print(thisDict)
```

```
{'Brand': 'Mahindra', 'Model': 'BE 6e Batman Edition', 'Year': 2025}
```

```python
thisDict={
    "Brand": "Mahindra",
    "Model": "BE 6e Batman Edition",
    "Year": 2025,
    "Year": 2026
}

print(thisDict)
```

```
{'Brand': 'Mahindra', 'Model': 'BE 6e Batman Edition', 'Year': 2026}
```

```python
x=thisDict["Model"]
print(x)

y=thisDict.get("Model")
print(y)

print(len(thisDict))

z=thisDict["Year"]
print(z)
```

```
BE 6e Batman Edition
BE 6e Batman Edition
3
2026
```

```python
x={
    'a':'b',
    'b':2,
    '2':6
}

print(x['2']) # 6
print(x[2]) # KeyError: 2
```

```
6

---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-30-da7ca0d41bc8> in <module>
      6
      7 print(x['2']) # 6
----> 8 print(x[2]) # KeyError: 2

KeyError: 2
```

```python
thisDict={
    "Brand": "Mahindra",
```

```
    "Model": "BE 6e Batman Edition",
    "Year": 2025
}

thisDict["Year"] = 2026
print(thisDict)
```

```
{'Brand': 'Mahindra', 'Model': 'BE 6e Batman Edition', 'Year': 2026}
```

```
for x in thisDict:
    print(x)
```

```
Brand
Model
Year
```

```
for x in thisDict:
    print(thisDict[x])
```

```
Mahindra
BE 6e Batman Edition
2026
```

```
for x in thisDict.values():
    print(x)
```

```
Mahindra
BE 6e Batman Edition
2026
```

```
for x in thisDict.keys():
    print(x)
```

```
Brand
Model
Year
```

```
for x, y in thisDict.items():
    print(x, ":", y)
```

```
Brand : Mahindra
Model : BE 6e Batman Edition
Year : 2026
```

```
thisDict={
    "Brand": "Mahindra",
    "Model": "BE 6e Batman Edition",
    "Year": 2026
}

print(thisDict)
```

```
{'Brand': 'Mahindra', 'Model': 'BE 6e Batman Edition', 'Year': 2026}
```

```
thisDict["Color"]="Matt Black"
print(thisDict)
```

```
{'Brand': 'Mahindra', 'Model': 'BE 6e Batman Edition', 'Year': 2026,
'Color': 'Matt Black'}
```

```
thisDict.popitem()
print(thisDict)
```

```
{'Brand': 'Mahindra', 'Model': 'BE 6e Batman Edition', 'Year': 2026}
```

```
thisDict.pop("Brand")
print(thisDict)
```

```
{'Model': 'BE 6e Batman Edition', 'Year': 2026}
```

```
thisDict={
    "Brand": "Mahindra",
    "Model": "BE 6e Batman Edition",
    "Year": 2026
}

del thisDict["Model"]
print(thisDict)
```

```
{'Brand': 'Mahindra', 'Year': 2026}
```

```
thisDict={
    "Brand": "Mahindra",
    "Model": "BE 6e Batman Edition",
    "Year": 2026
}

del thisDict
print(thisDict)
```

```
---------------------------------------------------------------------
-----
NameError                                 Traceback (most recent call
last)
<ipython-input-44-af22f29e0d2e> in <module>
      1 del thisDict
----> 2 print(thisDict)

NameError: name 'thisDict' is not defined
```

```
thisDict={
    "Brand": "Mahindra",
    "Model": "BE 6e Batman Edition",
```

```
    "Year": 2026
}

thisDict.clear()
print(thisDict)
```

```
{}
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 4

Part A (To be referred by students)

Sequences and Dictionary

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp4)

Aim: To understand and utilize Python collections (lists, tuples, sets, and dictionaries) and explore their applications in data science and statistics.

Problem Statement:

Solve the following computational problems by **recognizing** core Python data structures and **apply** them to implement programs

1. Create a list of the first 10 positive integers and print the sum of the even numbers in the list
2. Demonstrate the following operations on a number list using list functions
    a. insert 13 at position 4
    b. Sort the list in ascending order
    c. delete the last element
    d. remove 31
    e. Reverse the list
    f. Append one number to list
    g. extend the list with [20,30,40]
    h. print the number of elements in the list using while
3. Given the tuple (10, 20, 30, 40, 50), use slicing to print a new tuple containing only the last three elements.
4. Create a set of the numbers {1, 2, 3, 4, 5} and compute the intersection with {4, 5, 6, 7}; print the result.
5. Define a dictionary with the keys 'a', 'b', and 'c' and their respective values 1, 2, and 3, then update the value for key 'b' to 10 and print the updated dictionary.
6. Using a list comprehension, create a list of squares for the numbers from 1 to 5 and print the resulting list.
7. Create a tuple with elements 'cat', 'dog', and 'bird', then check and print whether 'dog' is in the tuple.
8. Given the list [1, 1, 2, 3, 5, 8, 13], convert it to a set and print the set's length.
9. Create two dictionaries {'A': [1, 2], 'B': [3, 4]} and {'B': [5, 6], 'C': [7, 8]}, then merge them into one dictionary and print the result.

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

10. For the list [3, 5, 7, 3, 5, 9, 3], create a dictionary to count the occurrences of each element and print the dictionary.
11. Create a set, add member(s) in a set and perform the following operations: intersection of sets, union of sets, set difference, symmetric difference, find length, maximum, minimum value in a set, and clear a set.
12. Create a dictionary with the day no as the key and the day as the value & display it. (ex. 1 – Monday, …)
13. Write a Python program to find the sum of all items in the dictionary.
14. Create a dictionary to keep students' marks, use students' sapid as the key. Perform the task mentioned below on this dictionary.
    a. Display all the keys
    b. Display all the values
    c. Take the sapid as the input and modify the grade given by the user
    d. Take the sapid from the user to remove that user from the dictionary
    e. Give 5 marks as a bonus to all the students and display the new marks
    f. Find the length of the dictionary using the len function
    g. Create a new copy of the dictionary using the copy method
15. Calculate total sales from a product-sales dictionary.
16. Find highest and lowest temperatures in [22, 25, 19, 30, 28, 32, 26].
17. Print the name of the student with the highest score from a list of tuples [(90, 'Alice'), (85, 'Bob'), (88, 'Charlie')].
18. Compute and print the cumulative sum of [2, 4, 6, 8, 10].
19. Create a list where numbers divisible by 3 become "Fizz", divisible by 5 become "Buzz", both become "FizzBuzz", for numbers 1 to 15.
20. Create a list of numbers from 1 to 100 divisible by both 2 and 5.
21. Write a program that takes a dictionary where each value is a list and converts it into a dictionary where each value is a set containing the unique elements from the original list.
22. Write a program that takes two dictionaries and returns a set of keys that are present in the first dictionary but not in the second.
23. Write a function that takes two sets of tuples and returns a tuple containing three sets:
    i. Elements that are in both sets.
    ii. Elements that are only in the first set.
    iii. Elements that are only in the second set.
24. Write a program that takes a list of sets and returns a single set that is the union of all the sets in the list.
25. Write a program that takes a list of tuples and rotates each tuple by a given number of positions. For example, rotating `(1, 2, 3, 4)` by 2 positions would result in `(3, 4, 1,2)`.
26. Write a program that takes a list of tuples where each tuple contains integers and returns the tuple with the highest sum of its elements.

Topic covered: List, Tuple, Set & Dictionary

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Learning Objective: Learner would be able to

1. Recognize core python data structures & apply them to solve problems
2. Solve the problems using sequences like list, tuple, set
3. Understand dictionary and solving problems using dictionary
4. Perform various operations (using functions) on sequences and dictionary

Theory:

Lists:

Lists are used to store multiple items in a single variable. Lists are created using square brackets. List items are ordered, changeable, and allow duplicate values. List can contain strings, integers, as well as objects. It can be used to implement stacks and queues.
List items are indexed, the first item has index [0], the second item has index [1] etc.

- **Create List:**

A tuple a list is created by placing elements inside square brackets [], separated by commas. A list can have any number of items and they may be of different types (integer, float, string, etc.). A list can also have another list as an item. This is called a nested list.

```
# list of integers
my_list = [1, 2, 3]
print(my_list)

# empty list
my_list = []
print(my_list)
# list with mixed data types
my_list = [1, "Hello", 3.4]
print(my_list)

# nested list
```

```
my_list = ["mouse", [8, 4, 6], ['a']]
print(my_list)
```

Output:

[1, 2, 3]
[]
[1, 'Hello', 3.4]
['mouse', [8, 4, 6], ['a']]

- **Access Items:**

  List Index:

  We can use the index operator [] to access an item in a list. In Python, indices start at 0. So, a list having 5 elements will have an index from 0 to. Python allows negative indexing for its sequences. The index of -1 refers to the last item, -2 to the second last item and so on. We can access a range of items in a list by using the slicing operator : .

```
my_list = ['p', 'r', 'o', 'b', 'e']

# first item
print(my_list[0])  # p

# third item
print(my_list[2])  # o

# fifth item
print(my_list[4])  # e

# Nested List
n_list = ["Happy", [2, 0, 1, 5]]
```

```
# Nested indexing
print(n_list[0][1])

print(n_list[1][3])

# Error! Only integer can be used for indexing
print(my_list[4.0])

# Negative indexing in lists
my_list = ['p','r','o','b','e']

# last item
print(my_list[-1])

# fifth last item
print(my_list[-5])

# List slicing in Python

my_list = ['p','r','o','g','r','a','m','i','z']

# elements from index 2 to index 4
print(my_list[2:5])
# elements from index 5 to end
print(my_list[5:])

# elements beginning to end
print(my_list[:])
```

Output:

```
p
o
e
a
5
Traceback (most recent call last):
File "<string>", line 21, in <module>
TypeError: list indices must be integers or slices, not float
e
p
['o', 'g', 'r']
['a', 'm', 'i', 'z']
['p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z']
```

- **Update Items:**

  We can use the assignment operator = to change an item or a range of items.

  We can add one item to a list using the append() method or add several items using the extend() method.

  We can also use + operator to combine two lists. This is also called concatenation.

  The * operator repeats a list for the given number of times.

  We can insert one item at a desired location by using the method insert() or insert multiple items by squeezing it into an empty slice of a list.

```
# Correcting mistake values in a list
odd = [2, 4, 6, 8]


# change the 1st item
odd[0] = 1
```

```
print(odd)

# change 2nd to 4th items
odd[1:4] = [3, 5, 7]
print(odd)

# Appending and Extending lists in Python
odd = [1, 3, 5]
odd.append(7)
print(odd)

odd.extend([9, 11, 13])
print(odd)

# Concatenating and repeating lists
odd = [1, 3, 5]
print(odd + [9, 7, 5])

print(["re"] * 3)
# Demonstration of list insert() method
odd = [1, 9]
odd.insert(1,3)
print(odd)

odd[2:2] = [5, 7]
print(odd)
```

Output:

[1, 4, 6, 8]

```
[1, 3, 5, 7]
[1, 3, 5, 7]
[1, 3, 5, 7, 9, 11, 13]
[1, 3, 5, 9, 7, 5]
['re', 're', 're']
[1, 3, 9]
[1, 3, 5, 7, 9]
```

- **Remove Item:**

  We can delete one or more items from a list using the Python del statement. It can even delete the list entirely.

```
# Deleting list items
my_list = ['p', 'r', 'o', 'b', 'l', 'e', 'm']

# delete one item
del my_list[2]

print(my_list)

# delete multiple items
del my_list[1:5]

print(my_list)

# delete the entire list
del my_list

# Error: List not defined
print(my_list)
```

Output:

['p', 'r', 'b', 'l', 'e', 'm']

['p', 'm']

Traceback (most recent call last):

File "<string>", line 21, in <module>

NameError: name 'my_list' is not defined

We can use remove() to remove the given item or pop() to remove an item at the given index.

The pop() method removes and returns the last item if the index is not provided. This helps us implement lists as stacks (first in, last out data structure).

And, if we have to empty the whole list, we can use the clear() method.

```
my_list = ['p','r','o','b','l','e','m']
my_list.remove('p')

# Output: ['r', 'o', 'b', 'l', 'e', 'm']
print(my_list)

# Output: 'o'
print(my_list.pop(1))

# Output: ['r', 'b', 'l', 'e', 'm']
print(my_list)

# Output: 'm'
print(my_list.pop())

# Output: ['r', 'b', 'l', 'e']
```

```
print(my_list)


my_list.clear()

# Output: []
print(my_list)
```

Output:

['r', 'o', 'b', 'l', 'e', 'm']
o
['r', 'b', 'l', 'e', 'm']
m
['r', 'b', 'l', 'e']
[]

We can also delete items in a list by assigning an empty list to a slice of elements. The sliced elements are removed from the original list.

```
my_list = ['p','r','o','b','l','e','m']
print(my_list[2:3])
my_list[2:3] = []
print(my_list)

print(my_list[2:5])
my_list[2:5] = []
print(my_list)
```

Output:

['o']
['p', 'r', 'b', 'l', 'e', 'm']

```
['b', 'l', 'e']
['p', 'r', 'm']
```

- **List Operations:**

1. List Membership Test:

We can test if an item exists in a list or not, using the keyword in.

```
my_list = ['p', 'r', 'o', 'b', 'l', 'e', 'm']

# Output: True
print('p' in my_list)

# Output: False
print('a' in my_list)

# Output: True
print('c' not in my_list)
```

Output:

True

False

True

2. Iterating through a List:

We can use a for loop to iterate through each item in a list.

```
for fruit in ['apple','banana','mango']:
    print("I like",fruit)
```

Output:

I like apple

```
I like banana
I like mango
```

- **List Comprehension:**

  List comprehension is an elegant and concise way to create a new list from an existing list in Python.

  A list comprehension consists of an expression followed by **for statement** inside square brackets.

```
pow2 = [2 ** x for x in range(10)]
print(pow2)

Output:
[1, 2, 4, 8, 16, 32, 64, 128, 256, 512]
```

  A list comprehension can optionally contain more for or **if statements**.

```
pow2 = [2 ** x for x in range(10) if x > 5]
print(pow2)
odd = [x for x in range(20) if x % 2 == 1]
print(odd)
z=[x+y for x in ['Python ','C '] for y in ['Language','Programming']]
print(z)

Output:
[64, 128, 256, 512]
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
['Python Language', 'Python Programming', 'C Language', 'C Programming']
```

Tuple:

A tuple is a sequence of **immutable** (cannot be changed) Python objects. Tuples use parentheses. Tuple are used to store sequence of python objects which are static in nature. Just like a list, a tuple contains a sequence of objects **in order** but once created a tuple, it cannot be changed anything about it. Tuple indices start from 0. Since tuples are indexed, they can have items with the same value.

- **Create a Tuple:**

  A tuple can have any number of items and they may be of different types (integer, float, list, string, etc.).

- **Create Tuple with One Item:**

  To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5);
tup3 = "a", "b", "c", "d";
print(tup1)
print(tup2)
print(tup3)
tup4 = ()
print(tup4)
tup4 = (50, );
print(tup4)
```

Output: Tuples are ordered and items cannot be changed once added.

('physics', 'chemistry', 1997, 2000)

(1, 2, 3, 4, 5)

('a', 'b', 'c', 'd')

()

(50,)

- **Access Items:**

  To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index.

```
tup1 = ('physics', 'chemistry', 1997, 2000)
tup2 = (1, 2, 3, 4, 5, 6, 7)
print ("tup1[0]: ", tup1[0])
print ("tup2[1:5]: ", tup2[1:5])
print(tup2[-1])
print(tup2[-6])
```

Output:

tup1[0]: physics

tup2[1:5]: (2, 3, 4, 5)

7

2

- **Update Items:**

  Tuples are immutable which means you cannot update or change the values of tuple elements. You are able to take portions of existing tuples to create new tuples.

```
tup1 = (12, 34.56)
tup2 = ('abc', 'xyz')
# Following action is not valid for tuples
# tup1[0] = 100
# So let's create a new tuple as follows
tup3 = tup1 + tup2
print (tup3)
```

Output:

(12, 34.56, 'abc', 'xyz')

- **Remove Item:**

  Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded. To explicitly remove an entire tuple, just use the del statement.

```
tup = ('physics', 'chemistry', 1997, 2000)
print (tup)
del (tup)
print (tup)
```

Output:

('physics', 'chemistry', 1997, 2000)

Traceback (most recent call last):

File "<string>", line 4, in <module>

NameError: name 'tup' is not defined

- **Tuple Operations:**

  3. Tuple Membership Test:

  We can test if an item exists in a tuple or not, using the keyword in.

```
# Membership test in tuple
my_tuple = ('a', 'p', 'p', 'l', 'e',)

# In operation
print('a' in my_tuple)
print('b' in my_tuple)

# Not in operation
print('g' not in my_tuple)
```

Output:

True

False

True

4. Iterating through a Tuple:

We can use a for loop to iterate through each item in a tuple.

```
# Using a for loop to iterate through a tuple
for name in ('John', 'Kate'):
    print("Hello", name)
```

Output:

Hello John

Hello Kate

5. Repetition:

Repeat the elements in a tuple for a given number of times using the * operator.

6. Concatenation:

Use + operator to combine two tuples. This is called concatenation.

7. Length:

Count the number of elements in the tuple.

```
# Concatenation
t = (1, 2, 3)
p =(4, 5, 6)
print(t + p)
```

```
# length
print(len(t))


# Repeat
# Output: ('Repeat', 'Repeat', 'Repeat')
print(("Repeat",) * 3)
```

Output:

(1, 2, 3, 4, 5, 6)

3

('Repeat', 'Repeat', 'Repeat')

Set:

A set is a collection which is **unordered** and **unindexed**. In Python sets are written with **curly brackets**. Every element is **unique (no duplicates)** and must be immutable (which cannot be changed). It can have any number of items and they may be of **different types (integer, float, tuple, string etc.)**. The sets in Python are typically used for mathematical operations like union, intersection, difference and complement etc.

- **Create a Set:**

```
thisset = {"apple", "banana", "cherry"}
print(thisset)
```

Output: Sets are unordered, so the items will appear in a random order.

{'banana', 'cherry', 'apple'}

- **Access Items:**

You cannot access items in a set by referring to an index, since sets are unordered the items has no index. But you can loop through the set items using a for loop, or ask if a specified value is present in a set, by using thein keyword.

```
thisset = {"apple", "banana", "cherry"}
print(thisset)

for x in thisset: print(x)

print("banana" in thisset)
```

Output:

{'cherry', 'banana', 'apple'}
cherry
banana
apple
True

- **Update Items:**

  Once a set is created, you cannot change its items, but you can add new items.

- **Add Items:**

  To add one item to a set use the add() method.

  To add more than one item to a set use the update() method.

```
thisset = {"apple", "banana", "cherry"}
thisset.add("orange")
print(thisset)
thisset.add("apple")
print(thisset)
```

Output:

{'apple', 'cherry', 'orange', 'banana'}

{'apple', 'cherry', 'orange', 'banana'}

Add multiple items to a set, using the update() method:

```
thisset = {"apple", "banana", "cherry"}
thisset.update(["orange", "mango", "grapes"])
print(thisset)
```

Output:

{'apple', 'cherry', 'orange', 'banana'}

- **Length of a Set:**

  To determine how many items a set has, use the len() method.

```
thisset = {"apple", "banana", "cherry"}
print(len(thisset))
```

Output:

3

- **Remove Item:**

  To remove an item in a set, use the remove(), or the discard() method.

```
# remove()
thisset = {"apple", "banana", "cherry"}
thisset.remove("banana")
print(thisset)

thisset.remove("banana")
print(thisset)
```

Output:

{'cherry', 'apple'}

```
Traceback (most recent call last):
  File "<string>", line 6, in <module>
KeyError: 'banana'
```

Note: If the item to remove does not exist, remove() will raise an error.

Note: If the item to remove does not exist, discard() will NOT raise an error.

```
# discard()
thisset = {"apple", "banana", "cherry"}
thisset.remove("banana")
print(thisset)
thisset.remove("banana")
print(thisset)

Output:
{'apple', 'cherry'}
{'apple', 'cherry'}
```

Note: Sets are unordered, so when using the gets removed. pop() method, you will not know which item that gets removed.

You can also use the pop(), method to remove an item, but this method will remove last item. Since set is unordered, so you will not know what item that gets. The return value of the pop() method is the removed item.

```
thisset = {"apple", "banana", "cherry"}
x = thisset.pop()
print(x)
print(thisset)

Output:
banana
```

```
{'cherry', 'apple'}
```

The clear() method empties the set:

```
thisset = {"apple", "banana", "cherry"}
thisset.clear()
print(thisset)
thisset.add("organge")
print(thisset)

Output:
set()
{'organge'}
```

The del() method will delete the set comple:

```
thisset = {"apple", "banana", "cherry"}
del thisset
print(thisset)

Output:
Traceback (most recent call last):
  File "<string>", line 5, in <module>
NameError: name 'thisset' is not defined
```

Dictionary:

A dictionary is a collection which is unordered, changeable and indexed. In Python dictionaries are written with curly brackets, and they have keys and values. It is a collection which is ordered (Python version 3.7 and above) and changeable (meaning that we can change, add or remove items after the dictionary has been created). No duplicate members.

- **Create a Dictionary:**

```
thisdict = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
print(thisdict)
```

Output: Dictionary are ordered.

{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

- **Access Items:**

  You can access the items of a dictionary by referring to its key name, inside square brackets.

```
# Get the value of the "model" key:
thisdict = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
x = thisdict["model"]
print(x)
```

Output:

Mustang

get() : There is also a method called get() that will give you the same result:

```
# Get the value of the "model" key:
thisdict = {
"brand": "Ford",
```

```
"model": "Mustang",
"year": 1964
}
x = thisdict.get("model")
print(x)
```

Output:

Mustang

- **Update and Add Items:**

  Dictionaries are mutable. We can add new items or change the value of existing items using an assignment operator.

  If the key is already present, then the existing value gets updated. In case the key is not present, a new (key: value) pair is added to the dictionary.

```
# Changing and Adding Dictionary elements:
thisdict = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
print(thisdict)
# update value
thisdict['year'] = 1966
print(thisdict)
# add item
thisdict["color"] = 'White'
print(thisdict)
```

Output:

{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

{'brand': 'Ford', 'model': 'Mustang', 'year': 1966}

{'brand': 'Ford', 'model': 'Mustang', 'year': 1966, 'color': 'White'}

- **Built-in functions of Dictionary:**

  all()

  Return True if all keys of the dictionary are True (or if the dictionary is empty).

  any()

  Return True if any key of the dictionary is true. If the dictionary is empty, return False.

  len()

  Return the length (the number of items) in the dictionary.

  cmp()

  Compares items of two dictionaries. (Not available in Python 3)

  sorted()

  Return a new sorted list of keys in the dictionary.

```
# Dictionary Built-in Functions
squares = {0: 0, 1: 1, 3: 9, 5: 25, 7: 49, 9: 81}

# Output: False
print(all(squares))

# Output: True
print(any(squares))

# Output: 6
print(len(squares))

# Output: [0, 1, 3, 5, 7, 9]
```

```
print(sorted(squares))
```

Output:

False

True

6

[0, 1, 3, 5, 7, 9]

- **Remove Item:**

  We can remove a particular item in a dictionary by using the **pop()** method. This method removes an item with the provided key and returns the value.

  The **popitem()** method can be used to remove and return an arbitrary (key, value) item pair from the dictionary.

  All the items can be removed at once, using the **clear()** method.

  We can also use the **del** keyword to remove individual items or the entire dictionary itself.

```
# Removing elements from a dictionary

# create a dictionary
squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# remove a particular item, returns its value
# Output: 16
print(squares.pop(4))

# Output: {1: 1, 2: 4, 3: 9, 5: 25}
print(squares)

# remove an arbitrary item, return (key,value)
# Output: (5, 25)
```

```
print(squares.popitem())

# Output: {1: 1, 2: 4, 3: 9}
print(squares)

# remove all items
squares.clear()
# Output: {}
print(squares)

# delete the dictionary itself
del squares

# Throws Error
print(squares)
```

Output:

```
16
{1: 1, 2: 4, 3: 9, 5: 25}
(5, 25)
{1: 1, 2: 4, 3: 9}
{}
Traceback (most recent call last):
  File "<string>", line 30, in <module>
NameError: name 'squares' is not defined
```

- **Looping through Dictionary:**

  A dictionary can be iterated using the for loop. If you want to get both keys and the values in the output. You just have to add the keys and values as the argument of the print

statement in comma separation. After each iteration of the for loop, you will get both the keys its relevant values in the output.

```
# The output shows the given key is related to the given value in the output.
dict={'Name':'Zara','Age':7,'Class':'First'}
for key, value in dict.items():
    print(key, '-',value)
```

Output:

Name - Zara

Age - 7

Class - First

- **Python Dictionary Comprehension:**

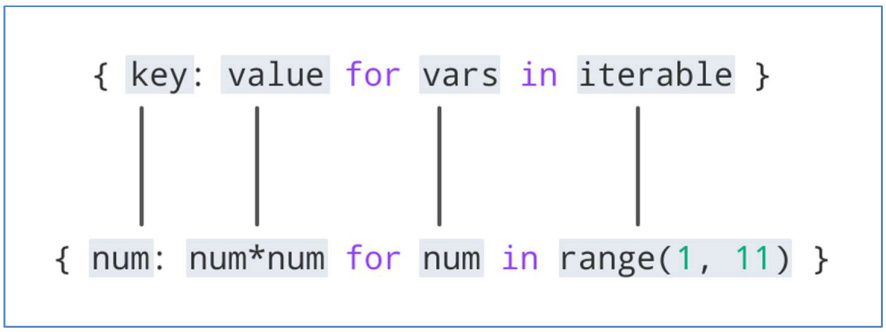
  Python Dictionary Comprehension helps to create a new dictionary from an iterable in Python.

  Dictionary comprehension consists of an expression pair (**key: value**) followed by a for statement inside curly braces {}.

  The minimal syntax for dictionary comprehension is:

```
dictionary = {key: value for vars in iterable}
```

Let's compare this syntax with dictionary comprehension from the above example.



```
# Dictionary Comprehension to create square of numbers from 0 to 5
squares = {x: x*x for x in range(6)}
print(squares)
```

Output:

{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Conditionals in Dictionary Comprehension:

We can further customize dictionary comprehension by adding conditions to it.

The syntax for dictionary comprehension with conditionals is:

```
dictionary = {key: true_value for vars in iterable   if  condition }
```

```
dictionary = {key: ( true_value if condition else false_value ) for vars in iterable}
```

```
# Apply dictionary comprehension to create squares of only odd numbers from 1 to 11

odd_squares = {x: x*x for x in range(11) if x % 2 == 1}
print(odd_squares)
```

Output: {1: 1, 3: 9, 5: 25, 7: 49, 9: 81}

```
# apply dictionary comprehension to pick up only items having even age values
original_dict = {'jack': 38, 'michael': 48, 'guido': 57, 'john': 33}

even_dict = {k: v for (k, v) in original_dict.items() if v % 2 == 0}
print(even_dict)
```

Output: {'jack': 38, 'michael': 48}

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 4

Part B (to be completed by students)

Sequences and Dictionary

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp4)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp4.pdf).
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
6. Upload the pdf on the web portal

Create a list of the first 10 positive integers and print the sum of the even numbers in the list

```
num=list(range(1,11))

list

sum=0
for i in num:
    if(i%2==0):
        sum+=i
print(sum)

30
```

Demonstrate the following operations on a number list using list functions a.insert 13 at position 4 b.Sort the list in ascending order c.delete the last element d.remove 31 e.Reverse the list f.Append one number to list g.extend the list with [20,30,40] h.print the number of elements in the list using while

```
num=list(range(1,11))
print(num)

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

num.insert(4,13)
print(num)

[1, 2, 3, 4, 13, 5, 6, 7, 8, 9, 10]

num.sort()
print(num)

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 13]

num.pop()
print(num)

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

num.remove(31)

---------------------------------------------------------------------------
-----
ValueError                                Traceback (most recent call
last)
<ipython-input-25-04da6c47085e> in <module>
----> 1 num.remove(31)

ValueError: list.remove(x): x not in list
```

```
num.sort(reverse=True)
print(num)
```

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

```
num.append(45)
print(num)
```

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 45]
```

```
l=[20,30,40]
num.extend(l)
print(num)
```

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 45, 20, 30, 40]
```

```
i=0
while i < len(num):
    print(num[i])
    i+=1
```

```
10
9
8
7
6
5
4
3
2
1
45
20
30
40
```

Given the tuple (10, 20, 30, 40, 50), use slicing to print a new tuple containing only the last three elements.

```
num=(10,20,30,40,50)
print(num[2:])
```

```
(30, 40, 50)
```

Create a set of the numbers {1, 2, 3, 4, 5} and compute the intersection with {4, 5, 6, 7}; print the result.

```
set1={1,2,3,4,5}
set2={4,5,6,7}
```

```
print(set1&set2)
```

```
{4, 5}
```

Define a dictionary with the keys 'a', 'b', and 'c' and their respective values 1, 2, and 3, then update the value for key 'b' to 10 and print the updated dictionary.

```
thisdict ={
    'a' : 1,
    'b' : 2,
    'c' : 3
}
print(thisdict)
dict['b']=10
print(thisdict)
```

```
{'a': 1, 'b': 2, 'c': 3}
{'a': 1, 'b': 2, 'c': 3}
```

Using a list comprehension, create a list of squares for the numbers from 1 to 5 and print the resulting list.

```
sq = [x**2 for x in range(1,6)]
print(sq)
```

```
[1, 4, 9, 16, 25]
```

Create a tuple with elements 'cat', 'dog', and 'bird', then check and print whether 'dog' is in the tuple.

```
animal=("cat","dog","bird")
print("dog" in animal)
```

```
True
```

Given the list [1,1,2,3,5,8,13], convert it to a set and print the set's length.

```
ls = [1,1,2,3,5,8,13]
print(ls)
s = set(ls)
print(s)
print(len(s))
```

```
[1, 1, 2, 3, 5, 8, 13]
{1, 2, 3, 5, 8, 13}
6
```

Create two dictionaries {'A': [1, 2], 'B': [3, 4]} and {'B': [5, 6], 'C': [7, 8]}, then merge them into one dictionary and print the result.

```
dict1 = {'A':[1,2], 'B':[3,4]}
dict2 = {'B':[5,6], 'C':[7,8]}
merged_dict = {**dict1, **dict2}
print(merged_dict)
```

```
{'A': [1, 2], 'B': [5, 6], 'C': [7, 8]}
```

For the list [3, 5, 7, 3, 5, 9, 3], create a dictionary to count the occurrences of each element and print the dictionary.

```
elements = [3, 5, 7, 3, 5, 9, 3]
counts = {}

for num in elements:
    if num in counts:
        counts[num] += 1
    else:
        counts[num]=1
print(counts)
```

```
{3: 3, 5: 2, 7: 1, 9: 1}
```

Name:- Aaryan N. Rodrane Roll No:- 5004 Div:- 5
batch:- A1 SAP ID:- 60018250070 branch:- A I & DS

Experiment No 4

* Title:- Sequences and dictionary

* Aim:- To understand and utilize Python collections (lists, tuples, sets and dictionaries) and explore their applications in data science and statistics.

* Theory:-

1) What is a list in Python? How is it different from an array in other programming languages?

⇒ A list is an ordered, mutable collection in Python that can store elements of different data types. Arrays in languages like C or Java require all elements to be of the same type and have a fixed size, whereas Python lists resize dynamically and accept mixed types.

Feature	Python list	Array (C / Java)
data types	Multiple types allowed	single type only
size	Dynamic	Fixed at declaration
Memory layout	Non contiguous	contiguous

code: data = [85, 90.5, "A", True]
print (data)

output: [85, 90.5, 'A', True]

2) Explain the concept of list Mutability with an example.

—> A list is mutable meaning its contents can be modified after creation. Elements can be updated, added or removed without replacing the entire list object.

code: marks = [70, 80, 90]
marks[0] = 95
print (marks)

output: [95, 80, 90]

3) Differentiate between append() and extend() methods.

=>

Method	Behaviour	Argument type
append()	adds a single element to the end	any object
extend()	unpacks and adds each element of an iterable	Iterable (list, tuple, etc.

Sundaram
FOR EDUCATIONAL USE

code:
```
a = [1, 2, 3]
a.append([4, 5])
print(a)

b = [1, 2, 3]
b.extend([4, 5])
print(b)
```

output:
```
[1, 2, 3, [4, 5]]
[1, 2, 3, 4, 5]
```

4) what is the difference between list.sort() and sorted(list)?

=>

feature	list.sort()	sorted(list)
Modifies original list	Yes (in place)	No
Return value	None	A new sorted list
works on	Lists only	Any iterable

code:
```
a = [3, 1, 2]
b = sorted(a)
print("sorted():", a, b)
a.sort()
print("list.sort():", a)
```

output:
```
sorted(): [3, 1, 2] [1, 2, 3]
list.sort(): [1, 2, 3]
```

Sundaram
FOR EDUCATIONAL USE

5) What is a tuple in Python? How is it different from list?

=> A tuple is an ordered, immutable collection defined using parentheses. Its elements cannot be changed once assigned.

Feature	Tuple	List
Mutability	immutable	Mutable
Syntax	()	[]
Performance	Faster	Slower
Use case	fixed/constant data	Dynamic data

Code :-
```
cords = (10.5, 20.3)
print(cords[0])
```

Output :- 10.5

6) Why are tuples called immutable? What are the advantages of immutability?

=> Tuples are immutable because their elements cannot be modified, added or deleted after creation. Any attempt to do so raises a TypeError.

The advantage of tuple is that they are safe for storing data that should not change, they can be used as dictionary keys (since they are hashable), and they iterate faster than

Sundaram
FOR EDUCATIONAL USE

lists due to lower memory overhead

code :-
```
t = (1, 2, 3)
t[0] = 99
```

output :- TypeError: 'tuple' object does not support item assignment

7) Can a tuple contain mutable objects like lists? Explain with an example.

=> Yes, the tuple itself remains immutable (its references cannot be reassigned), but a mutable object stored inside it can still be modified through its own methods

code :-
```
t = (1, [2, 3], 4)
t[1].append(99)
print(t)
```

output :- (1, [2, 3, 99], 4)

8) What is a set in Python and what are its main properties?

=> A set is an unordered collection of unique, hashable elements. It is defined using curly braces or the set() constructor. Sets support mathematical operations such as union, intersection and difference.

Sundaram
FOR EDUCATIONAL USE

Main properties :-

- unordered (no indexing)
- duplicates not allowed
- elements must be immutable

Code :-
```
s = {10, 20, 30, 20, 10}
print(s)
print(type(s))
```

Output :-
```
{10, 20, 30}
<class 'set'>
```

Q) How are sets different from lists and tuples?

⇒

Feature	set	List	Tuple
order	unordered	ordered	ordered
duplicates	not allowed	allowed	allowed
Mutability	Mutable	Mutable	Immutable
Indexing	Not supported	supported	supported

Code :-
```
lst = [1, 2, 2, 3]
tpl = (1, 2, 2, 3)
st = {1, 2, 2, 3}
print(lst, tpl, st)
```

output :- [1, 2, 2, 3] (1, 2, 2, 3) {1, 2, 3}

Sundaram
FOR EDUCATIONAL USE

10) Why are duplicates not allowed in a set?

⇒ sets use a hash table internally. Each element is stored at a position determined by its hash value. When a element with an identical hash (ie, or equal value) is inserted, it replaces the existing entry rather than creating a new one. This mechanism enforces uniqueness automatically.

code :-
```
s = set([4,4,5,5,6])
print(s)
```

output : {4,5,6}

11) What is the difference between remove() and discard() methods in sets?

⇒

method	when element is not found
remove()	Raises KeyError
discard()	Does nothing, no error raised

code :-
```
s = {1,2,3}
s.discard(99)
print(s)
s.remove(99)
```

output :- {1,2,3}
KeyError: 99

Sundaram FOR EDUCATIONAL USE

(2) what is a dictionary in Python? How is it different from a list?

⇒ A dictionary is an unordered collection of key value pairs. Elements are accessed using keys rather than numeric indices.

Feature	Dictionary	List
Access Method	By key	By index
Element structure	Key value pairs	single values
Key requirement	must be immutable	Not applicable

code :- student = {"name": "Raj", "Marks": 88}
print (student ["name"])

output :- Raj

(3) Explain the concept of key-value pairs in dictionaries.

⇒ Every entry in a dictionary has two parts: a key and a value. The key acts as a unique identifier and the value holds the associated data. Keys must be unique within a dictionary, but values can repeat.

Sundaram
FOR EDUCATIONAL USE

code :- course = {
"subject" : "Python",
"code" : "DJS23SLPC303",
"credits" : 2
}
print(course["subject"], "-->", course["code"])

output :- Python --> DJS23SLPC303

(a) Why must dictionary keys be immutable (like strings, numbers, tuples)?

=> Python uses a hash table to store dictionary keys. Hashing requires the value of an object to remain constant. Mutable objects like lists have no stable hash, so attempting to use them as keys raises a TypeError.

code :- d = {}
d[(1,2)] = "valid"
print(d)
d[[1,2]] = "invalid"

output :- {(1,2) : 'valid'}
TypeError : unhashable type : 'list'

Sundaram
FOR EDUCATIONAL USE

(5) Differentiate between get() and direct key access (eg. d['key']).

⟹

Feature	get()	d['key']
Key not found	Returns none or a specified default	Raises KeyError
safety	Safe for unknown Keys	Risky without prior check
Default value	can be specified	Not applicable

code:-
```
d={"a":100}
print(d.get("b",0))
print(d["b"])
```

output:-
```
0
KeyError: 'b'
```

*Conclusion:-

This experiment covers the four built in collection types in Python, namely lists, tuples, sets, and dictionaries, each differing in mutability, ordering, and element constraints. Lists and tuples handle ordered sequence while sets enforce uniqueness through hashing and dictionaries provide fast key based data access. Selecting the appropriate collection type based on these properties is essential for writing correct and efficient data science programs.

Sundaram
FOR EDUCATIONAL USE

Numpy (Numerical Python)

```
import numpy as np
```

Creating Numpy Array and Studying its types

```
# normal array
list1=[2,4,6,8]
array = np.array(list1)
print(array)
```

```
[2 4 6 8]
```

```
# array of 0 and 1
array0 = np.zeros(4)
array1 = np.ones(4)
print(array0,array1)
```

```
[0. 0. 0. 0.] [1. 1. 1. 1.]
```

```
# creating array with range
arr1 = np.arange(5)
arr2 = np.arange(1,6)
arr3 = np.arange(1,11,2)
print(arr1,arr2,arr3)
```

```
[0 1 2 3 4] [1 2 3 4 5] [1 3 5 7 9]
```

```
# random array generator
arr = np.random.rand(5)
print(arr)
```

```
[0.83177336 0.10457029 0.03207396 0.96526122 0.61636389]
```

```
# random integer array generator
arr = np.random.randint(1,101,size=5)
print(arr)
```

```
[39 55 62 68 42]
```

```
# random integer array generator with seed
np.random.seed(0)
arr = np.random.randint(1,101,size=5)
print(arr)
```

```
[45 48 65 68 68]
```

```
# generate 5 random number uniformly distributed distributed between
0.0 and 1.0
```

```
arr = np.random.uniform(0.0,1.0,size=5)
print(arr)
```

```
[0.84725174 0.6235637  0.38438171 0.29753461 0.05671298]
```

```
arr = np.empty(5)
print(arr)
```

```
[0.84725174 0.6235637  0.38438171 0.29753461 0.05671298]
```

```
# 1d array
arr1d = np.zeros(5)
print(arr1d)
print(arr1d.shape)
```

```
[0. 0. 0. 0. 0.]
(5,)
```

```
# 2d array
arr2d = np.zeros((2,3))
print(arr2d)
print(arr2d.shape)
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
(2, 3)
```

```
# 3d array
arr3d = np.zeros((2,3,4))
print(arr3d)
print(arr3d.shape)
```

```
[[[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]

 [[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]]
(2, 3, 4)
```

```
# 4d array
arr4d = np.zeros((2,3,4,5))
print(arr4d)
print(arr4d.shape)
# (no of 3d arrays,no of 2d arrays,rows,columns)
```

```
[[[[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]
```

```
  [[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]

  [[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]]


 [[[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]

  [[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]

  [[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]]]
(2, 3, 4, 5)
```

```
arr = np.array([[1,2,3],[4,5,6]])
print(arr.ndim)
print(len(arr))
print(arr.shape)
print(arr.dtype.name)
print(arr.astype(int))
print(arr.dtype)
```

```
2
2
(2, 3)
int32
[[1 2 3]
 [4 5 6]]
int32
```

```
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])
print("Number of Dimension : ",arr.ndim)
print("Size of array : ",arr.size)
print("Shape of array : ",arr.shape)
print("Data type of elements : ",arr.dtype)
print("Size of each elements in bytes : ",arr.itemsize)
```

```
Number of Dimension :  2
Size of array :  9
Shape of array :  (3, 3)
Data type of elements :  int32
Size of each elements in bytes :  4
```

```
arr2 = np.array([[1,2,3],[4,5,6],[7,8,9]],dtype=np.int64)
print(arr2.itemsize)
print(arr2.data)
```

```
8
<memory at 0x0000024E90E92EA0>
```

```
arr2d = np.array([[1,2,3,4],[5,6,7,8]])
print(arr2d)
```

```
[[1 2 3 4]
 [5 6 7 8]]
```

```
arr4d = np.array([[[1,2,3,4],[5,6,7,8],[9,10,11,12]],[[1,2,3,4],
[5,6,7,8],[9,10,11,12]]])
print(arr4d)
```

```
[[[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]

 [[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]]
```

```
arr = np.zeros((2,3))
print("2d array")
print(arr)
```

```
2d array
[[0. 0. 0.]
 [0. 0. 0.]]
```

```
arr = np.zeros((2,3,4))
print("3d array")
print(arr)
```

```
3d array
[[[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]

 [[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]]
```

```
arr = np.ones((2,3,4))
print("3d array")
print(arr)
```

```
3d array
[[[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]

 [[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]]
```

```
numpy_array = np.full((2,2),5)
print(numpy_array)
```

```
[[5 5]
 [5 5]]
```

```
numpy_array_3d = np.random.rand(2,2,2)
print(numpy_array_3d)
```

```
[[[0.27265629 0.47766512]
  [0.81216873 0.47997717]]

 [[0.3927848  0.83607876]
  [0.33739616 0.64817187]]]
```

```
numpy_array_3d = np.empty((2,2,2))
print(numpy_array_3d)
```

```
[[[0.27265629 0.47766512]
  [0.81216873 0.47997717]]

 [[0.3927848  0.83607876]
  [0.33739616 0.64817187]]]
```

```
carr1 = np.array([1+2j,2+3j,3+8j])
print(carr1)
print(carr1.dtype)
print(carr1.itemsize)
```

```
[1.+2.j 2.+3.j 3.+8.j]
complex128
16
```

```
farr = carr1.astype('float')
print(farr)
```

```
[1. 2. 3.]
```

```
C:\Users\djsce.student\AppData\Local\Continuum\anaconda3\lib\site-packages\ipykernel_launcher.py:1: ComplexWarning: Casting complex values to real discards the imaginary part
  """Entry point for launching an IPython kernel.
```

Numpy array Indexing

starts with 0

```
arr1 = np.array([1,3,5,9])
print(arr1[0])
print(arr1[2])
print(arr1[-1])
```

```
1
5
9
```

```
arr1[1]=11
arr1[3]=13
print(arr1)
```

```
[ 1 11  5 13]
```

```
print(arr1[-2])
```

```
5
```

20th August

```
arr1 = np.array([[1, 3, 5], [7, 9, 2], [4, 6, 8]])
print("output : ",arr1[2, 1])
```

```
6
```

```
print("2nd Row: ",arr1[1,:])
```

```
2nd Row:  [7 9 2]
```

```
print("3rd column: ",arr1[:,2])
```

```
3rd column:  [5 2 8]
```

```
arr2 = np.array([[[1,2,3,4],[5,6,7,8],[9,10,11,12]],[[13,14,15,16],[17,18,19,20],[21,22,23,24]]])
print("element : ",arr2[1,2,1])
print("row : ",arr2[1,2,:])
print("column : ",arr2[1,:,1])
print("what to call it ? : ",arr2[:,2,1])
```

```
element :  22
row :  [21 22 23 24]
column :  [14 18 22]
what to call it ? :  [10 22]
```

```
arr = np.array([1,3,5,7,9,2,4,6,8])
print(arr[2:6])
print(arr[0:8:2])
print(arr[3:])
```

```
[5 7 9 2]
[1 5 9 4]
[7 9 2 4 6 8]
```

```
arr = np.array([10,20,30,40,50])
print(arr[:2])
print(arr[2:])
print(arr[::-1])
```

```
[10 20]
[30 40 50]
[50 40 30 20 10]
```

```
# changing by indexing
arr[3:]=100
print(arr)
```

```
[ 10  20  30 100 100]
```

```
arr[:3]=20
print(arr)
```

```
[ 20  20  20 100 100]
```

```
arr[::2]=0
print(arr)
```

```
[  0  20   0 100   0]
```

```
arr = np.array([10,20,30,40,50])
print(arr[-3:])
print(arr[-5:-2])
print(arr[-1::-2])
```

```
[30 40 50]
[10 20 30]
[50 30 10]
```

```
arr = np.array([[1,3,5,7],[9,11,13,15]])
print(arr[:2,:2])
```

```
[[ 1  3]
 [ 9 11]]

print(arr[:2,2:])

[[ 5  7]
 [13 15]]

arr = np.array([1,3,5,7,2,4,6,8])
result = np.reshape(arr,(2,4))
print(result)

[[1 3 5 7]
 [2 4 6 8]]

result = np.reshape(result,(2,2,2))
print(result)

[[[1 3]
  [5 7]]

 [[2 4]
  [6 8]]]

print(np.reshape(arr,(2,2,2)))

[[[1 3]
  [5 7]]

 [[2 4]
  [6 8]]]

# flattening or array
result = np.reshape(result,-1)
print(result)

[1 3 5 7 2 4 6 8]

# adding 2 arrays
arr1 = np.array([1,3,5,7])
arr2 = np.array([8,6,4,2])
print(arr1+arr2)

[9 9 9 9]

# mean,median,mode
marks = np.array([76,78,81,66,85])
print("mean : ",np.mean(marks))
print("median : ",np.median(marks))
print("minimum : ",np.min(marks))
print("maximum : ",np.max(marks))
```

```
mean :  77.2
median :  78.0
minimum :  66
maximum :  85
```

```
# less then , greater then , equal to
print("arr1 < arr2 : ",arr1 < arr2)
print("arr1 > arr2 : ",arr1 > arr2)
print("arr1 == arr2 : ",arr1 == arr2)
print("arr1 <= arr2 : ",arr1 <= arr2)
print("arr1 <= arr2 : ",np.less_equal(arr1,arr2))
print("arr1 >= arr2 : ",arr1 >= arr2)
```

```
arr1 < arr2 :  [ True  True False False]
arr1 > arr2 :  [False False  True  True]
arr1 == arr2 :  [False False False False]
arr1 <= arr2 :  [ True  True False False]
arr1 <= arr2 :  [ True  True False False]
arr1 >= arr2 :  [False False  True  True]
```

```
# column-wise axis=0
# row-wise axis=1 similar for mean,median,comparison
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(np.sum(arr,axis=0))
print(np.sum(arr,axis=1))
```

```
[12 15 18]
[ 6 15 24]
```

```
#slicing
arr=np.array([10,20,30,40,50,60,70,80])
print(arr[1:6])
```

```
[20 30 40 50 60]
```

```
#negative slicing
numbers=np.array([5,10,15,20,25,30,35,40])
print(numbers[-5:-1])
```

```
[20 25 30 35]
```

```
#slicing extract 2nd and 3d row
arr = np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]])
print(arr[1:3,:])
```

```
[[ 5  6  7  8]
 [ 9 10 11 12]]
```

```
#slicing extract 1sr and 3d column
arr = np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]])
print(arr[:,0:3:2])
```

```
[[ 1  3]
 [ 5  7]
 [ 9 11]
 [13 15]]
```

```
#numpy array 1 to 12 matrix 3x4
arr = np.arange(1,13)
arr = np.reshape(arr,(3,4))
print(arr)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

```
#numpy array 1 to 24 in 3d shape
arr = np.arange(1,25)
arr = np.reshape(arr,(2,3,4))
print(arr)
```

```
[[[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]

 [[13 14 15 16]
  [17 18 19 20]
  [21 22 23 24]]]
```

```
#numpy array 1 to 20 in 4x5 and 5x4 shape
arr = np.arange(1,21)
arr = np.reshape(arr,(4,5))
print(arr)
arr = np.reshape(arr,(5,4))
print(arr)
```

```
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]]
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]
 [17 18 19 20]]
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 5

Part A (To be referred by students)

NumPy

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp5)

Aim: To understand and apply basic and advanced NumPy operations and built-in matrixfunctions, focusing on their application in data science and machine learning tasks.

Problem Statement:

Solve the following computational problems by **recognizing** python data structures of NumPy and **apply** them to implement programs

1. Create a 3x3 matrix of random integers between 1 and 10 and find the total sum of its elements.
2. Generate a 4x4 matrix of random numbers and compute the average of all its elements.
3. Create a 2x3 matrix and find the sum of all elements in the second row.
4. Generate a 5x5 matrix of random integers and determine the highest value in each column.
5. Create a 4x4 matrix and calculate the sum of its main diagonal elements.
6. Generate a 3x3 matrix and find the smallest value among all its elements.
7. Create a 4x4 matrix of random integers between 1 and 20 and compute the sum of all elements that are greater than 10.
8. Generate a 5x5 matrix of random numbers and find the mean of the maximum values of each row.
9. Create a 3x3 matrix and calculate the difference between the sum of the elements in the first row and the sum of the elements in the last row.
10. Generate a 6x6 matrix of random integers and replace all values below the median with zero.
11. Create a 4x4 matrix and compute the sum of elements in the upper triangular part of the matrix.
12. Generate a 3x4 matrix of random values and find the standard deviation of the elements in the third column.

Topic covered: Numpy

Learning Objective: Learner would be able to

1. Learn how to import packages in python
2. Features and available functions of Numpy

3. Create multi-dimensional data structures and manipulate them

Theory:

- NumPy is 'Numerical Python package consisting of multidimensional array (ndarray) objects and a collection of routines for processing of array.

#Numpy with one dimension import numpy as np a = np.array([1,2,3]) print(a) O/P - [1, 2, 3]	# more than one dimensions import numpy as np a = np.array([[1, 2], [3, 4]]) print(a) O/P - [[1, 2] [3, 4]]

NumPy's array class is called ndarray. The important attributes of an ndarray object are:
ndarray.ndim - the number of axes (dimensions) of the array.
ndarray.shape - the dimensions of the array. This is a tuple of integers indicating the size of the array in each dimension. For a matrix with n rows and m columns, shape will be (n,m). The length of the shape tuple is therefore the number of axes, ndim.
ndarray.size -the total number of elements of the array. This is equal to the product of the elements of shape.
ndarray.dtype -an object describing the type of the elements in the array. One can create or specify dtype's using standard Python types. Additionally, NumPy provides types of its own. numpy.int32, numpy.int16, and numpy.float64 are some examples.
ndarray.itemsize - the size in bytes of each element of the array. For example, an array of elements of type float64 has itemsize 8 (=64/8), while one of type complex32 has itemsize 4 (=32/8). It is equivalent to ndarray.dtype. itemsize.
ndarray.data - the buffer containing the actual elements of the array. Normally, we won't need to use this attribute because we will access the elements in an array using indexing facilities.
Example –

import numpy as np a = np.arange(15).reshape(3, 5) print(a) array([[0, 1, 2, 3, 4], [5, 6, 7, 8, 9], [10, 11, 12, 13, 14]]) print(a.shape) (3, 5) print(a.ndim) 2 print(a.dtype.name) 'int64'	print(a.itemsize) 8 print(a.size) 15 print(type(a)) <class 'numpy.ndarray'> print(b = np.array([6, 7, 8]) print(b) array([6, 7, 8]) print(type(b)) <class 'numpy.ndarray'>

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Array Creation There are several ways to create arrays. For example, you can create an array from a regular Python list or tuple using the array function

import numpy as np a = np.array([2, 3, 4]) print(a) array([2, 3, 4]) print(a.dtype) dtype('int64') print(b = np.array([1.2, 3.5, 5.1]) print(b.dtype) dtype('float64')	print(a = np.array(1, 2, 3, 4) # WRONG Traceback (most recent call last): ... TypeError: array() takes from 1 to 2 positional arguments but 4 were given print(a = np.array([1, 2, 3, 4]) # RIGHT	b = np.array([(1.5, 2, 3), (4, 5, 6)]) print(b) array([[1.5, 2. , 3.], [4, 5 , 6]]) **The type of the array can also be explicitly specified at creation time:** print(c = np.array([[1, 2], [3, 4]], dtype=**complex**) print(c) array([[1.+0.j, 2.+0.j], [3.+0.j, 4.+0.j]])

To create sequences of numbers, NumPy provides the arange function which is analogous to the Python built-in range, but returns an array.
print(np.arange(10, 30, 5))
array([10, 15, 20, 25])
print(np.arange(0, 2, 0.3)) # it accepts float arguments
array([0. , 0.3, 0.6, 0.9, 1.2, 1.5, 1.8])

Universal Functions
NumPy provides familiar mathematical functions such as sin, cos, and exp

B = np.arange(3) print(B) array([0, 1, 2]) print(np.exp(B)) array([1. , 2.71828183, 7.3890561])	print(np.sqrt(B)) array([0. , 1. , 1.41421356]) print(C = np.array([2., -1., 4.]) print(np.add(B, C)) array([2., 0., 6.])

Changing the **shape** of an array

a = np.floor(10 * rg.random((3, 4)))
print(a)
array([[3., 7., 3., 4.],
[1., 4., 2., 2.],
[7., 2., 4., 9.]])
print(a.shape)
(3, 4)

The shape of an array can be changed with various commands. Note that the following three commands all return a modified array, but do not change the original array:

```
print( a.ravel()) # returns the array, flattened
array([3., 7., 3., 4., 1., 4., 2., 2., 7., 2., 4., 9.])
print( a.reshape(6, 2)) # returns the array with a modified shape
array([[3., 7.],
[3., 4.],
[1., 4.],
[2., 2.],
[7., 2.],
[4., 9.]])
print( a.T) # returns the array, transposed
array([[3., 1., 7.],
[7., 4., 2.],
[3., 2., 4.],
[4., 2., 9.]])
print( a.T.shape)
(4, 3)
print( a.shape)
(3, 4)
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 5

Part B (to be completed by students)

Numpy

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp5)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp5.pdf).
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
6. Upload the pdf on the web portal

```python
import numpy as np

#sum of element of 3 x 3
arr3d = np.random.randint(1,11,size=(3,3))
print(arr3d)
Sum = np.sum(arr3d)
print("Sum of all Elements : ",Sum)
```

```
[[ 2  5  2]
 [ 4  9  4]
 [ 2  6 10]]
Sum of all Elements :  44
```

```python
#avg of element of 4 x 4
arr4d = np.random.rand(4,4)
print(arr4d)
avg = np.mean(arr4d)
print("Average of all Elements : ",avg)
```

```
[[0.31008133 0.30666187 0.03657576 0.03351542]
 [0.40332796 0.04365721 0.43461072 0.13389531]
 [0.86378715 0.21645602 0.27535305 0.07617275]
 [0.8475716  0.15397093 0.68304676 0.56588243]]
Average of all Elements :  0.33653539215167105
```

```python
#sum of element of 2 x 3
arr2x3 = np.random.rand(2,3)
print(arr2x3)
Sum = np.sum(arr2x3[:,1])
print("Sum of all Elements of 2nd row : ",Sum)
```

```
[[0.62572517 0.34119676 0.20156804]
 [0.80421055 0.28441033 0.87016062]]
Sum of all Elements of 2nd row :  0.6256070889323414
```

```python
# find highest element in 5x5
arr5x5 = np.random.randint(1,10001,size=(5,5))
print(arr5x5)
Max = np.max(arr5x5)
print("Max among all Elements : ",Max)
```

```
[[1508 4004 1511 6881 7283]
 [ 530 7680  213 7470 5627]
 [1296 2579 2639 3159 1268]
 [1358 4418 7124  343 2146]
 [2876 8985  344  278 2659]]
Max among all Elements :  8985
```

```python
# find trace in 4x4
arr4x4 = np.random.rand(4,4)
print(arr4x4)
```

```
tr = np.trace(arr4x4)
print("sum of all diagonal elements : ",tr)
```

```
[[0.38969828 0.32538589 0.29082692 0.38099127]
 [0.87866076 0.99828621 0.93642776 0.59968742]
 [0.3412576  0.03916221 0.23970246 0.04576408]
 [0.48057372 0.27434913 0.94587712 0.26478667]]
sum of all diagonal elements :  1.8924736319021438
```

```
# find highest element in 3x3
arr3x3 = np.random.rand(3,3)
print(arr3x3)
Min = np.min(arr3x3)
print("Minimum among all Elements : ",Min)
```

```
[[0.38374901 0.23625982 0.61691999]
 [0.14352691 0.03455422 0.05900485]
 [0.64946168 0.99616589 0.63250666]]
Minimum among all Elements :  0.03455422138235509
```

```
# sum all elements > 10 in 3X3
arr3x3 = np.random.randint(1,21,size=(3,3))
print(arr3x3)
Sum = np.sum(arr3x3[arr3x3 > 10])
print("Sum of all Elements of 2nd row : ",Sum)
```

```
[[17  5  1]
 [13  1  4]
 [10  6 20]]
Sum of all Elements of 2nd row :  50
```

```
# Generate a 5x5 matrix of random numbers and find the mean of the
maximum values of each row.
arr5x5 = np.random.randint(1, 101, size=(5, 5))
print(arr5x5)
row_maxima = np.max(arr5x5, axis=1)
print("Row Max :", row_maxima)
mean_of_maxima = np.mean(row_maxima)
print("Mean of Max :", mean_of_maxima)
```

```
[[37 73 95 60 10]
 [21 89 84 70 62]
 [73 34 42 12 89]
 [24 71  6 20 25]
 [ 2 63 97 58 16]]
Row Maxima: [95 89 89 71 97]
Mean of Maxima: 88.2
```

```
# Create a 3x3 matrix and calculate the difference between the sum of
the elements in the first row and
# the sum of the elements in the last row.
```

```
arr3x3 = np.random.randint(1, 21, size=(3, 3))
print(arr3x3)
sum_first_row = np.sum(arr3x3[0, :])
sum_last_row = np.sum(arr3x3[-1, :])
difference = sum_first_row - sum_last_row
print("Difference between sum of first and last row : ", difference)
```

```
[[ 9 19 15]
 [18  5  5]
 [14  9 16]]
Difference between sum of first and last row :  4
```

```
# Generate a 6x6 matrix of random integers and replace all values
below the median with zero.
arr6x6 = np.random.randint(1, 101, size=(6, 6))
print(arr6x6)
median_val = np.median(arr6x6)
print("Median value : ",median_val)
arr6x6[arr6x6 < median_val] = 0
print("Matrix after replacing values below median with zero:\n",
arr6x6)
```

```
[[18 53  2 18 50 81]
 [12 16 15 92 29 77]
 [58 48 29 36 57 80]
 [33 69 16 18 48  9]
 [16 51 16 96 70 96]
 [56 25 44 28  7 47]]
Median value :  40.0
Matrix after replacing values below median with zero:
 [[ 0 53  0  0 50 81]
 [ 0  0  0 92  0 77]
 [58 48  0  0 57 80]
 [ 0 69  0  0 48  0]
 [ 0 51  0 96 70 96]
 [56  0 44  0  0 47]]
```

```
# Create a 4x4 matrix and compute the sum of elements in the upper
triangular part of the matrix.
arr4x4 = np.random.randint(1, 101, size=(4, 4))
print(arr4x4)
upper_tri_elements = arr4x4[np.triu_indices(4)]
sum_upper_tri = np.sum(upper_tri_elements)
print("Sum of elements in the upper triangular part : ",
sum_upper_tri)
```

```
[[90 84 62 47]
 [ 8 97 73 54]
 [97 85 61 23]
```

```
 [33 95 90 53]]
Sum of elements in the upper triangular part :  644
```

```
# Generate a 3x4 matrix of random values and find the standard
deviation of the elements in the third col.
arr3x4 = np.random.randint(1, 101, size=(3, 4))
print(arr3x4)
third_column_std = np.std(arr3x4[:, 2])
print("Standard deviation : ", third_column_std)
```

```
[[19 65 16 88]
 [12 74 19 34]
 [69 18 89 82]]
Standard deviation :  33.7276675083759
```

Name :- Aaryan N. Raorane Roll No :- 5004

Branch :- AI-DS batch : A1 SAP ID : 60018250070

Experiment No 5 : Numpy

Aim : To understand and apply basic and advanced Numpy operations and built in matrix functions, focusing on their application in data science and machine learning tasks.

Theory :

1. What is array slicing ? Provide an example where you extract a sub array from a 2D Numpy array.

=> Array slicing is a technique in Numpy that allows extraction of a contiguous portion of an array by specifying index ranges along each dimension. For a 2D array, the syntax is

```
array[row_start:row_end, col_start:col_end]
```

The indices follow Python conventions, where the start index is inclusive and the end index is exclusive. Slicing returns a view, not copy, meaning modifications to the slice affect the original array

Example :

```
import numpy as np
arr = np.array([[10,20,30,40],
                [50,60,70,80],
                [90,100,110,120]])
sub = arr[0:2, 1:3]
print("original array:")
print(arr)
```

Sundaram FOR EDUCATIONAL USE

```
print ("Extracted subarray:")
print (sub)
```

output:

```
Original Array:
[[10 20 30 40]
 [50 60 70 80]
 [90 100 110 120]]
Extracted Subarray:
[[20 30]
 [60 70]]
```

2. Describe element-wise operations in Numpy. Why are they useful in data science applications?

=> Elementwise operations in Numpy are arithmetic or mathematical operations applied independently to each corresponding pair of elements across two arrays of the same shape. Numpy vectorizes these computations internally using optimized C routines, which eliminates the need for explicit Python loops and result in significantly faster execution. In data science, elementwise operations are widely used for feature scaling, computing residuals between predicted and actual value, applying transformations such as squaring or taking square roots across an entire dataset, and performing column-wise arithmetic on numerical data.

Sundaram®
FOR EDUCATIONAL USE

Example :-

```
import numpy as np
a= np.array([2,4,6,8])
b= np.array([1,3,5,7])

print("a+b:", a+b)
print("a*b:", a*b)
print("a**2:", a**2)
print("np.sqrt(a):", np.sqrt(a))
```

output :-

```
a+b: [3 7 11 15]
a*b: [2 12 30 56]
a**2: [4 16 36 64]
np.sqrt(a): [1.41421356 2.
             2.44948974 2.82842712]
```

3. What is the purpose of using ~~like np~~ functions like np.zeros(), np.ones(), and np.empty()? When would you use each?

⇒ These three functions are used to create Numpy arrays of a specified shape with requiring explicit data input. Each serves a distinct initialization purpose.

i) np.zeros(shape) creates an array where every element is initialized to 0.0 It is used when a zero ~~beta~~ baseline is required, such as initializing weight matrices before training a neural network or creating placeholder arrays for accumulating sums.

Sundaram
FOR EDUCATIONAL USE

ii) np.ones(shape) creates an array where every element to 1.0. It is commonly used to construct bias vectors, mask arrays, or initial arrays where a multiplicative identity is needed.

iii) np.empty(shape) creates an array without initializing its values. The memory is allocated but the contents are arbitary (garbage values from previously stored data). It is the fastest of the three fn and is appropriate only when the array will be completely overwritten before use, making it useful in performance critical numerical routines.

Example: import numpy as np

```
z = np.zeros((2,3))
o = np.ones((2,3))
e = np.empty((2,3))
print("np.zeros((2,3)):\n", z)
print("np.ones((2,3)):\n", o)
print("np.empty((2,3)):\n", e)
```

Output:
```
np.zeros((2,3)):
[[0. 0. 0.]
 [0. 0. 0.]]
np.ones((2,3)):
[[1. 1. 1.]
 [1. 1. 1.]]
```

Sundaram FOR EDUCATIONAL USE

np.empty((2,3)):

[[1.0094556 6e-315 0.00000000e+000
0.00000000e+000]
[0.00000000e+000 0.00000000e+000
0.00000000e+000]]

4. Explain the difference between np.arange() and np.linspace(). Provide a use-case for each function.

=>

Feature	np.range()	np.linspace()
control parameter	step size (interval between values)	Number of evenly spaced points.
Endpoint included	No (stop is exclusive)	Yes (stop is included by default)
output data type	Inferred from arguments (int or float)	Always float 64
Primary use case	Iterating over integer index ranges	Generating uniform sample points for plots or interpolation.
Predictability of Endpoint	Endpoint may be omitted due to floating point rounding	Endpoint is always guaranteed.

Sundaram®
FOR EDUCATIONAL USE

Use case for np.arange(): Generating indices for iteration, such as creating a sequence of pixel values from 0 to 255 in step of 1.

Use case for np.linspace(): Generating uniformly distributed x values for plotting a mathematical function or creating equally spaced sample points for signal processing.

Example: import numpy as np

```
r = np.arange(0,10,2)
print("np.arange(0,10,2):", r)

l = np.linspace(0,1,5)
print("np.linspace(0,1,5):", l)
```

Output: np.arange(0,10,2): [0 2 4 6 8]
np.linspace(0,1,5): [0 0.25 0.5 0.75 1.]

5. How does reshaping an array work in Numpy? What will happen if the total number of elements does not match the new shape?

=> Reshaping in Numpy refers to changing the dimensional structure of an array without altering any of its underlying data values. The total number of elements must remain identical before and after the reshape operation. Numpy simply reinterprets the flat memory

Sundaram®
FOR EDUCATIONAL USE

layout of the array under new shape specification. If the product of the new shape dimension does not equal the total number of Element in the orignal array, Numpy raises a ValueError at runtime. For example, an array of 12 elements can be reshaped to (3,4), (2, 6), (4,3), or (12,), but attempting to reshape it to (3, 5) which requires 15 elements will cause an Error

Example: import numpy as np

```
a = np.arange(1,13)
print("orignal:", a)

b = a.reshape(3, 4)
print("Reshaped to (3,4):\n", b)

try:
    c = a.reshape(3,5)
except ValueError as e:
    print("Error:", e)
```

output: orignal: [1 2 3 4 5 6 7 8 9 10 11 12]
Reshaped to (3, 4):
[[1 2 3 4]
[5 6 7 8]
[9 10 11 12]]
error: cannot reshape array of size 12 into shape (3,5)

Sundaram
FOR EDUCATIONAL USE

6. What is matrix multiplication in Numpy, and how is it different from element wise multiplication?

=>

Aspect	Elementwise Multiplication	Matrix multiplication
operator / function	* operator	@ operator or np.matmul()
operation performed	Multiplies each element at position (i,j) in A with the element at the same position (i,j) in B	computes the dot product of rows of A with columns of B
Shape requirement	Both arrays must have identical shapes	Number of columns in A must equal number of rows in B.
Result shape	Same as input arrays	depends on outer dimensions of operands
Mathematical meaning	No standard linear algebra equivalent	standard matrix product from linear algebra

Sundaram
FOR EDUCATIONAL USE

Typical use case	masking, scaling, Hadamard product in ML	Linear transformation, neural network forward pass, solving systems of equations.

Example: import numpy as np

```
A = np.array([[1,2], [3,4]])
B = np.array([[5,6], [7,8]])

print("Elementwise (A * B):\n", A * B)
print("Matrix Multiplication (A @ B):\n", A @ B)
```

Output:
```
Elementwise (A * B):
[[5 12]
 [21 32]]
Matrix Multiplication (A @ B):
[[19 22]
 [43 50]]
```

7. Explain what broadcasting is in Numpy. Provide an example where broadcasting simplifies operations on arrays of different shapes.

⟹ Broadcasting is the mechanism in Numpy that allows arithmetic operations to be performed between arrays of different shapes without explicitly copying the data; Numpy automatically stretches the smaller

Sundaram
FOR EDUCATIONAL USE

array along dimensions where it has size 1, so that it matches the shape of the larger array during computation. The rules governing broadcasting are applied by comparing dimension from the trailing end. Two dimensions are compatible if they are equal or if one of them is 1. If neither condition holds, Numpy raises a ValueError. Broadcasting avoids the memory overhead of creating expanded copies and allows concise, readable code.

Example: adding a row vector of shape (3,) to a matrix of shape (3,3). The row vector is broadcast across all three rows of the matrix

```
import numpy as np

matrix = np.array([ [1,2,3],
                    [4,5,6],
                    [7,8,9] ])
row = np.array([10, 20, 30])
result = matrix + row
print("Matrix + Row (Broadcasting):\n", result)
```

Output:
```
Matrix + Row (Broadcasting):
[[11 22 33]
 [14 25 36]
 [17 28 39]]
```

Sundaram
FOR EDUCATIONAL USE

8. Describe how you would normalize a dataset using Numpy. Why is normalization important in machine learning tasks?

=> Normalization scales all values in a dataset to fixed range, typically [0, 1], using the min-max formula:

$$normalized = (x - x_{min}) / (x_{max} - x_{min})$$

This transformation ensures that smallest value in the dataset maps to 0 and the largest maps to 1, with all other values scaled proportionally in between. Normalization is critical in machine learning for several reasons. Algorithms based on distance computation such as K-Nearest neighbours and support vector machines are heavily biased toward features with larger numerical scales if normalization is not applied. Gradient descent based algorithm converge significantly faster when all input features are on a comparable scale, avoiding situations where the loss surface is elongated along certain dimensions.

Example:

```
import numpy as np
data = np.array([10.0, 20.0, 30.0, 40.0, 50.0])
normalized = (data - data.min()) / (data.max() - data.min())
print("Original data:", data)
print("Normalized data:", normalized)
```

Sundaram
FOR EDUCATIONAL USE

output:

Original data: [10. 20. 30. 40. 50.]
Normalized data: [0. 0.25 0.5 0.75 1.]

9. What is a covariance Matrix, and how does numpy help in computing it? Explain its importance in data science.

⟹ A covariance Matrix is a square symmetric matrix where the entry at position (i, j) represents the covariance between the ith and jth features of a dataset. The diagonal entries represent the variance of each individual feature, while the off-diagonal entries indicate how two features vary together. A positive covariance between features i and j means they tend to increase and decrease together, while a negative covariance means they vary in opposite directions. Numpy computes the covariance matrix using np.cov(), where each row of the input matrix is treated as one variable (feature) and each column as one observation. The function applies the sample covariance formula, dividing by N minus 1 rather than N to produce an unbiased estimate. The covariance matrix is central to several data science techniques. In principal component analysis (PCA), the eigenvectors of the covariance

Sundaram
FOR EDUCATIONAL USE

matrix define the principal components, enabling dimensionality reduction in multivariate statistics, it is used for linear discriminant analysis, Mahalanobis distance computation, and gaussian distribution modelling

Example:

```
import numpy as np

data = np.array([[2,3,5],[7,4,6],[1,8,9]])
cov = np.cov(data)
print("Dataset:\n", data)
print("Covariance:\n", cov)
```

Output:

```
Dataset:
[[2 3 5]
 [7 4 6]
 [1 8 9]]
Covariance Matrix:
[[2.33333333 -0.33333333  5.5       ]
 [-0.33333333 2.33333333 -4.5       ]
 [5.5        -4.5         19.       ]]
```

Sundaram®
FOR EDUCATIONAL USE

conclusions:

This experiment showed that Numly offers an efficient framework for numerical computing, covering array creation, slicing, reshaping, and vectorized mathematical operations that eliminate explicit loops and boost performance. Matrix operations like multiplication and covariance computation confirmed Numly's accurate implementation of linear algebra fundamentals underlying ML algorithms such as PCA and linear regression. Techniques like min-max normalization and covariance computation, applied to real datasets, established clear link between Numly's API and core data science workflows.

Sundaram®
FOR EDUCATIONAL USE